

[Lecture Notebook] Data Visualization

MGs3701 Data Mining, Spring 2025

Chad (Chungil Chae)

Mon, 26 May 2025

Table 3.2

```
housing.df <- read.csv("data/BostonHousing.csv")
head(housing.df, 9)
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	LSTAT	MEDV
1	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	4.98	24.0
2	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	9.14	21.6
3	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	4.03	34.7
4	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	2.94	33.4
5	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	5.33	36.2
6	0.02985	0.0	2.18	0	0.458	6.430	58.7	6.0622	3	222	18.7	5.21	28.7
7	0.08829	12.5	7.87	0	0.524	6.012	66.6	5.5605	5	311	15.2	12.43	22.9
8	0.14455	12.5	7.87	0	0.524	6.172	96.1	5.9505	5	311	15.2	19.15	27.1
9	0.21124	12.5	7.87	0	0.524	5.631	100.0	6.0821	5	311	15.2	29.93	16.5
CAT..MEDV													
1	0												
2	0												
3	1												
4	1												
5	1												
6	0												
7	0												
8	0												
9	0												

26 **Figure 3.1**27 **Amtrak ridership by year**

```
Amtrak.df <- read.csv("data/Amtrak data.csv")
```

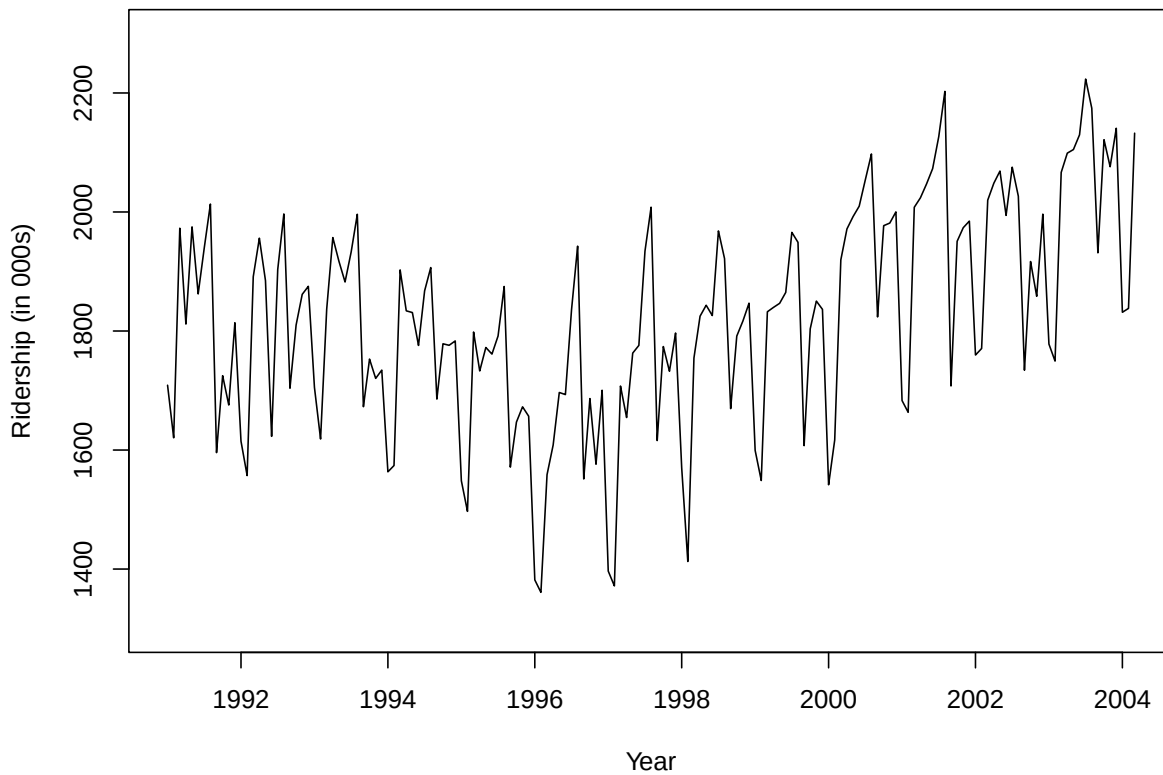
```
# use time series analysis  
library(forecast)
```

```
28 Registered S3 method overwritten by 'quantmod':
```

```
29   method      from
```

```
30   as.zoo.data.frame zoo
```

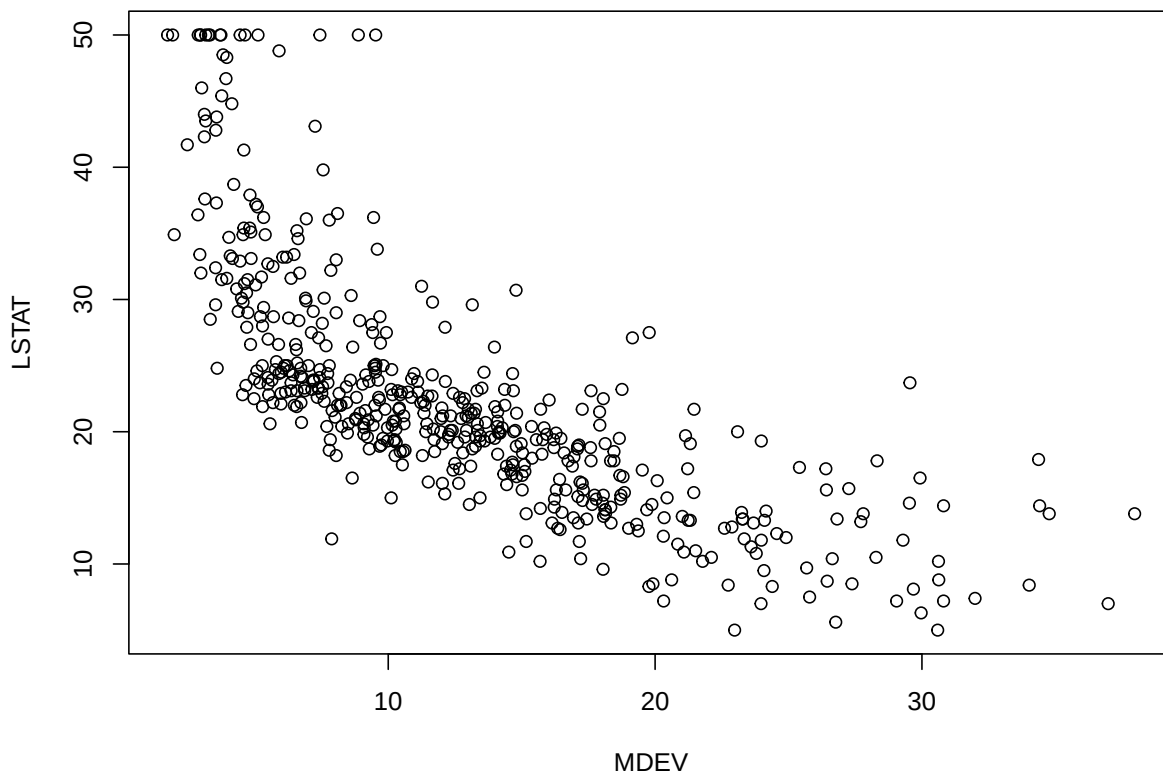
```
ridership.ts <- ts(Amtrak.df$Ridership, start = c(1991, 1), end = c(2004, 3), freq = 12)  
plot(ridership.ts, xlab = "Year", ylab = "Ridership (in 000s)", ylim = c(1300, 2300))
```



31

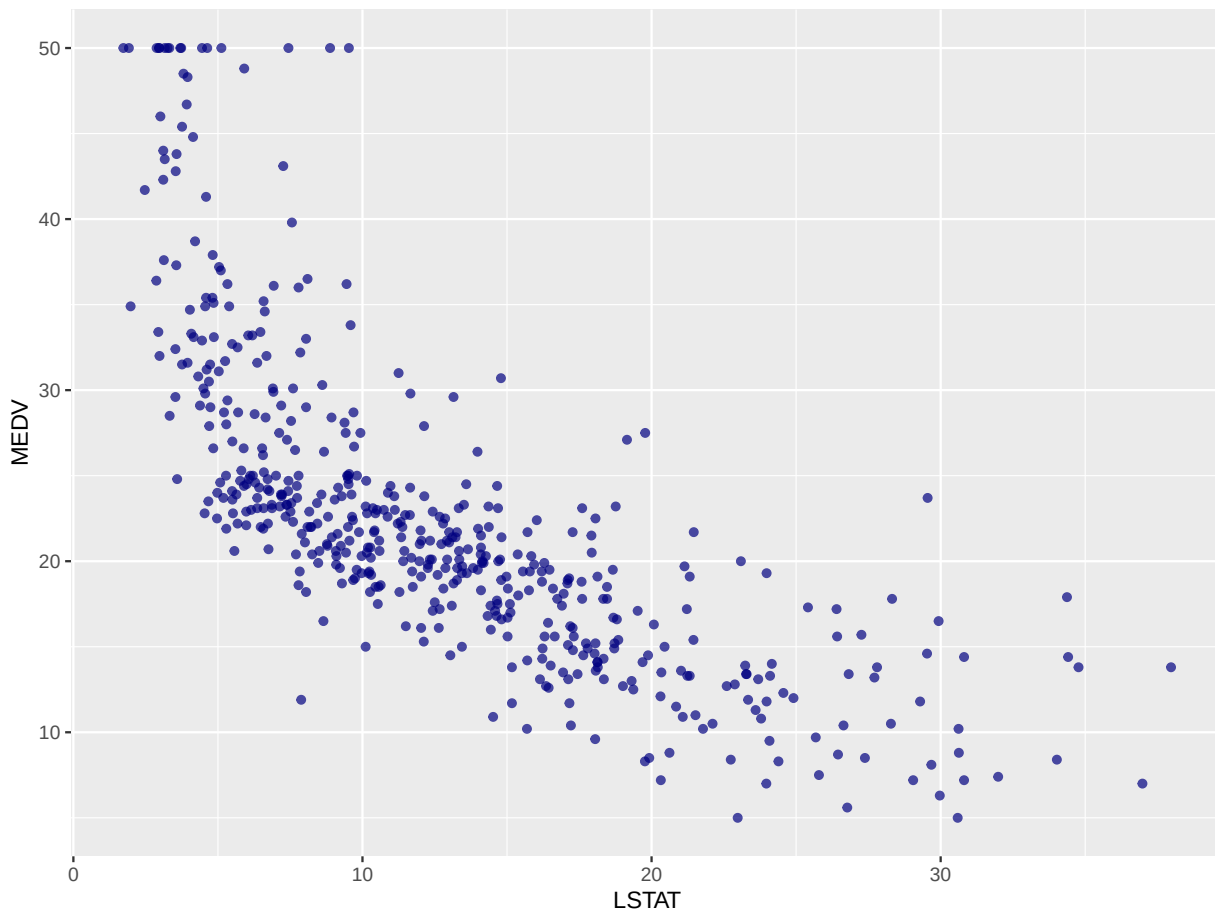
32 **Boston housing data**

```
housing.df <- read.csv("data/BostonHousing.csv")  
  
## scatter plot with axes names  
plot(housing.df$MEDV ~ housing.df$LSTAT, xlab = "MDEV", ylab = "LSTAT")
```



33

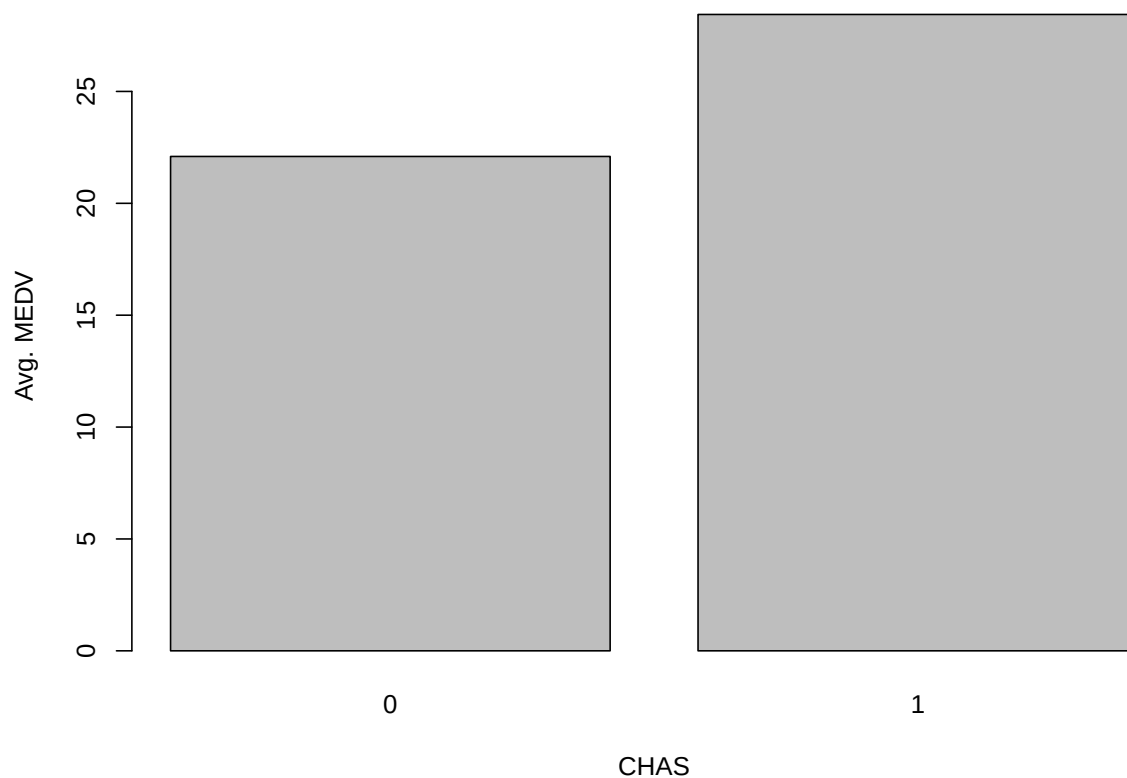
```
# alternative plot with ggplot  
library(ggplot2)  
ggplot(housing.df) + geom_point(aes(x = LSTAT, y = MEDV), colour = "navy", alpha = 0.7)
```



34

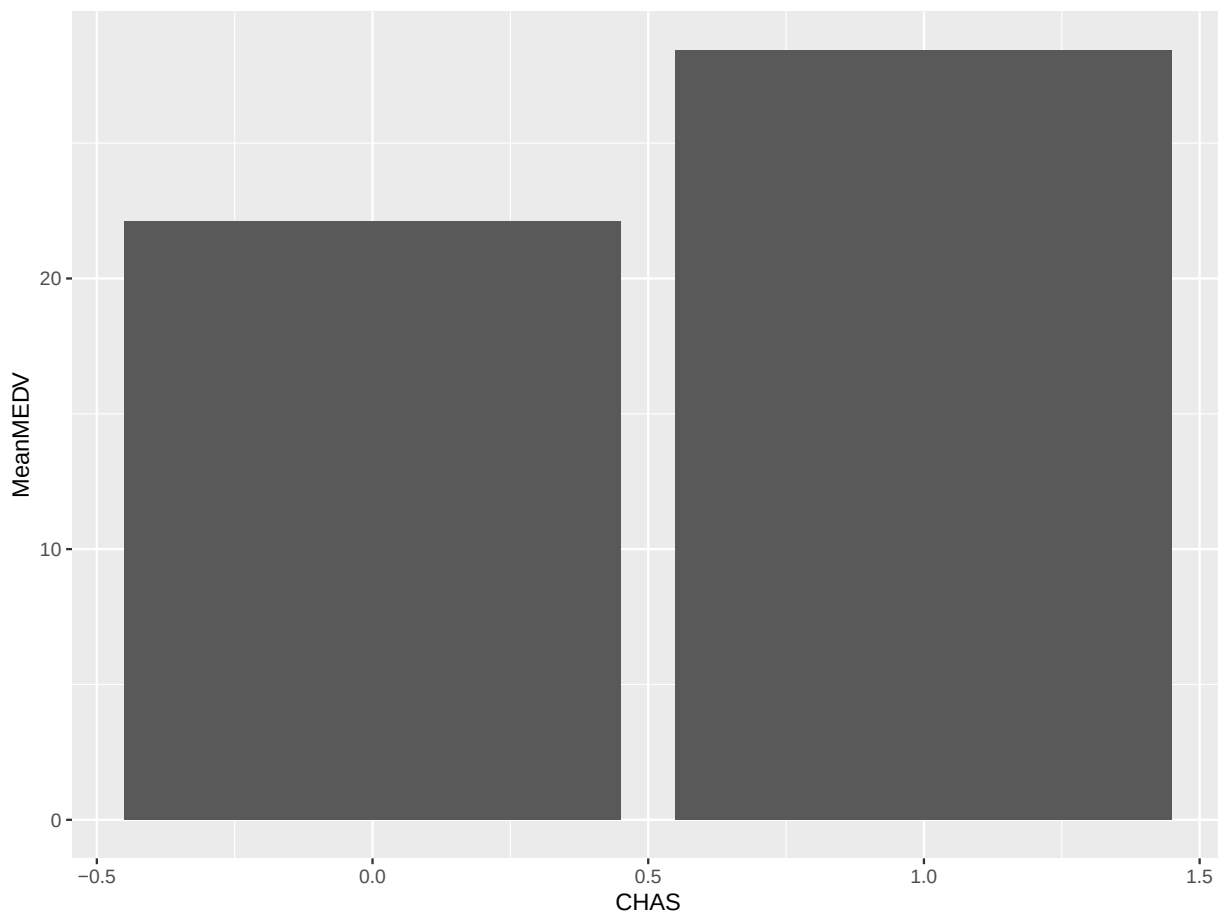
35 **barchart of CHAS vs. mean MEDV**

```
# compute mean MEDV per CHAS = (0, 1)
data.for.plot <- aggregate(housing.df$MEDV, by = list(housing.df$CHAS), FUN = mean)
names(data.for.plot) <- c("CHAS", "MeanMEDV")
barplot(data.for.plot$MeanMEDV, names.arg = data.for.plot$CHAS,
        xlab = "CHAS", ylab = "Avg. MEDV")
```



36

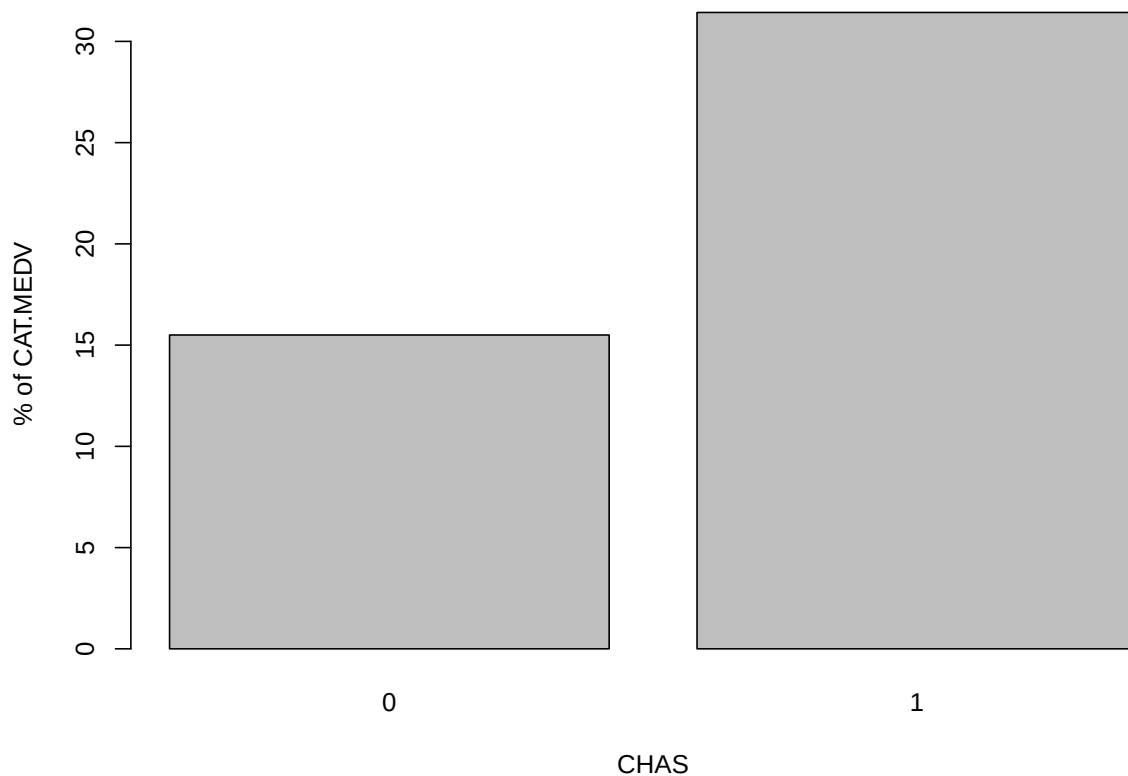
```
# alternative plot with ggplot  
ggplot(data.for.plot) + geom_bar(aes(x = CHAS, y = MeanMEDV), stat = "identity")
```



37

38 **barchart of CHAS vs. % CAT.MEDV**

```
data.for.plot <- aggregate(housing.df$CAT..MEDV, by = list(housing.df$CHAS), FUN = mean)
names(data.for.plot) <- c("CHAS", "MeanCATMEDV")
barplot(data.for.plot$MeanCATMEDV * 100, names.arg = data.for.plot$CHAS,
        xlab = "CHAS", ylab = "% of CAT.MEDV")
```

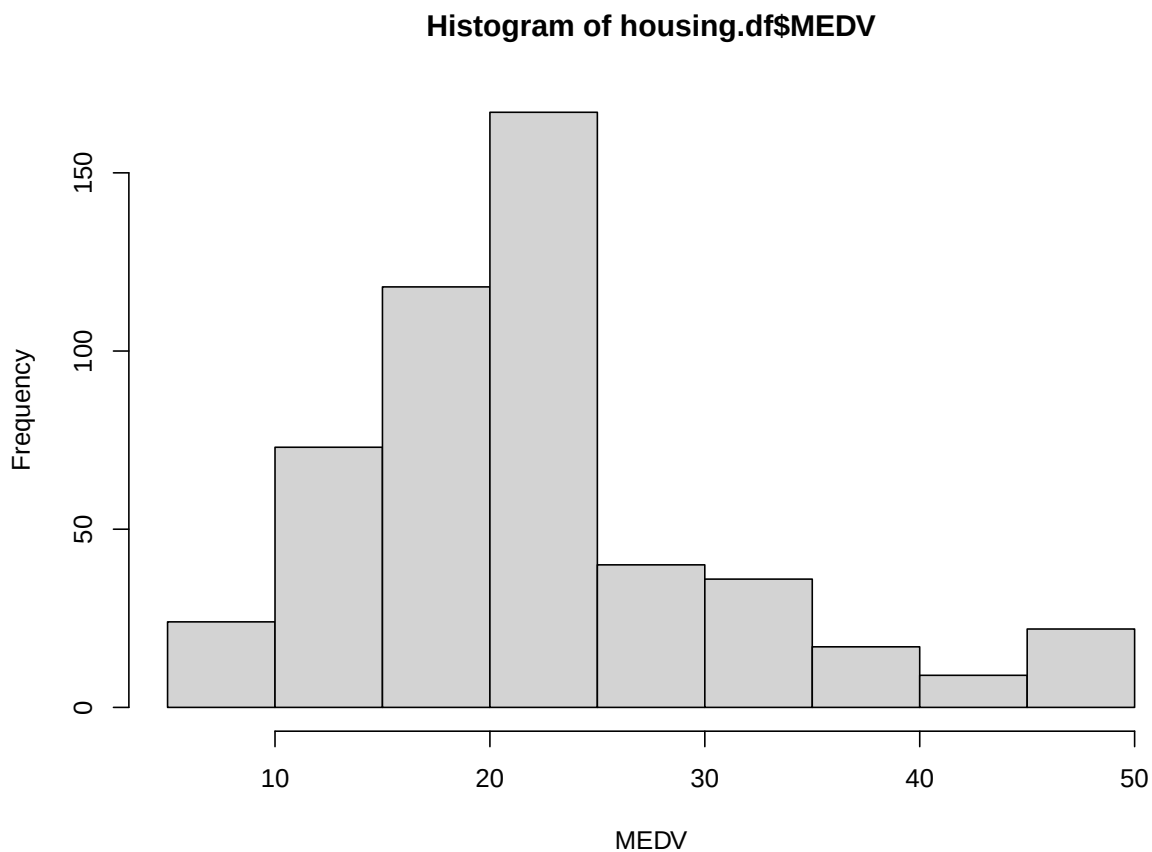


39

40 Figure 3.2

41 histogram of MEDV

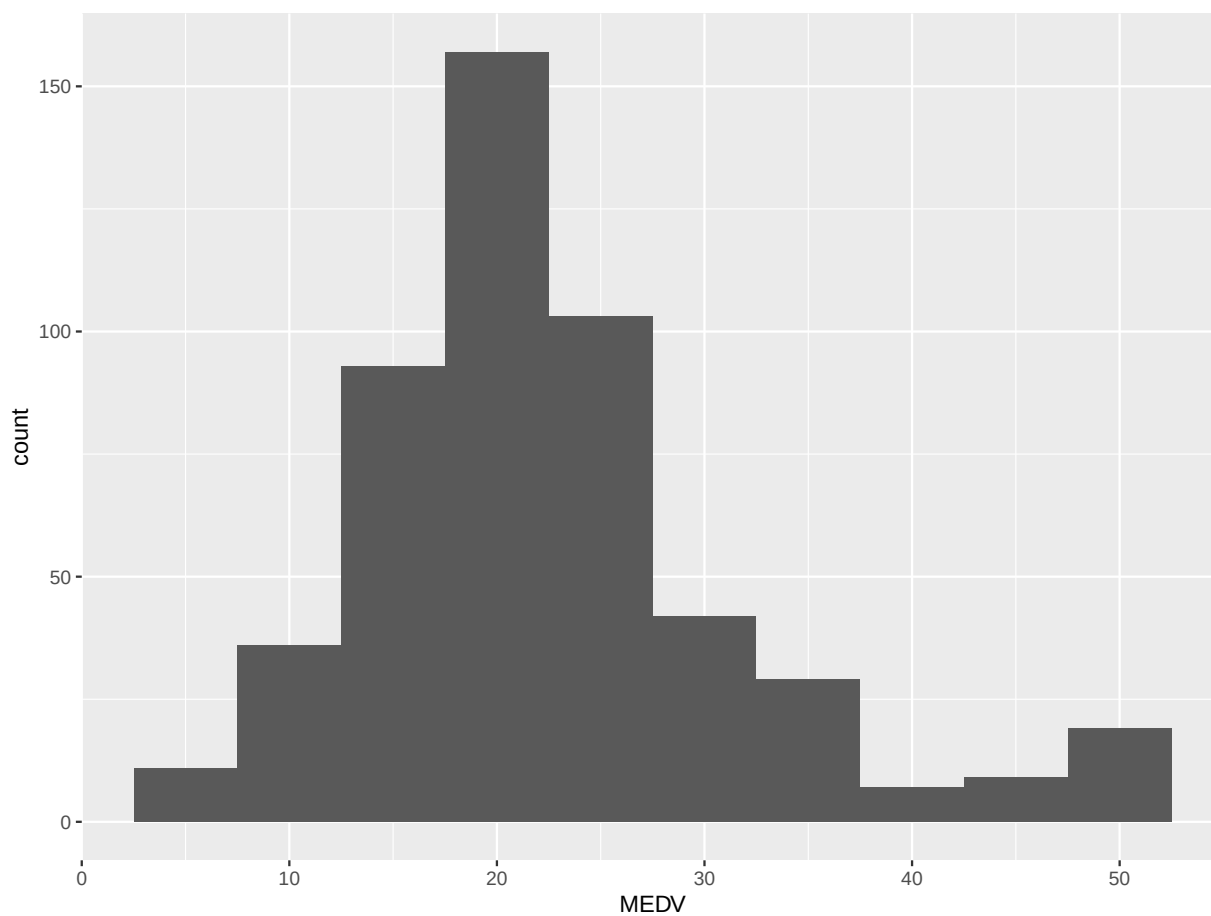
```
hist(housing.df$MEDV, xlab = "MEDV")
```



42

43 alternative plot with ggplot

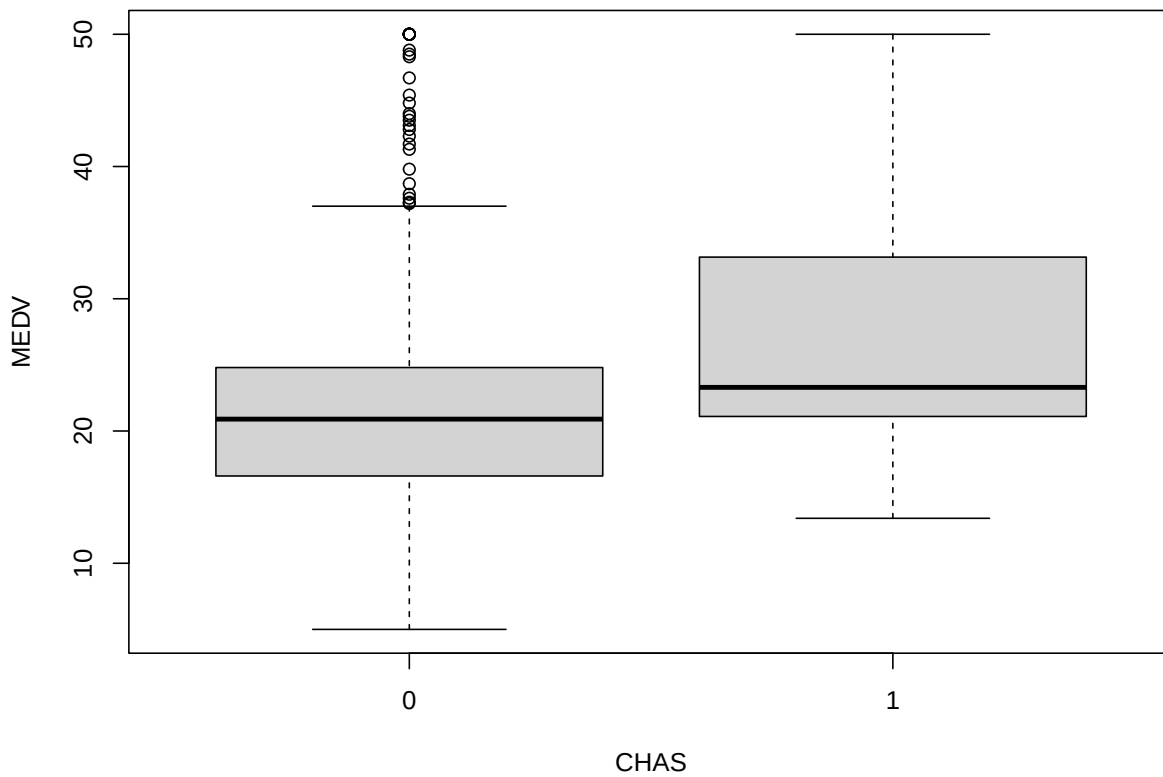
```
library(ggplot2)
ggplot(housing.df) + geom_histogram(aes(x = MEDV), binwidth = 5)
```

44

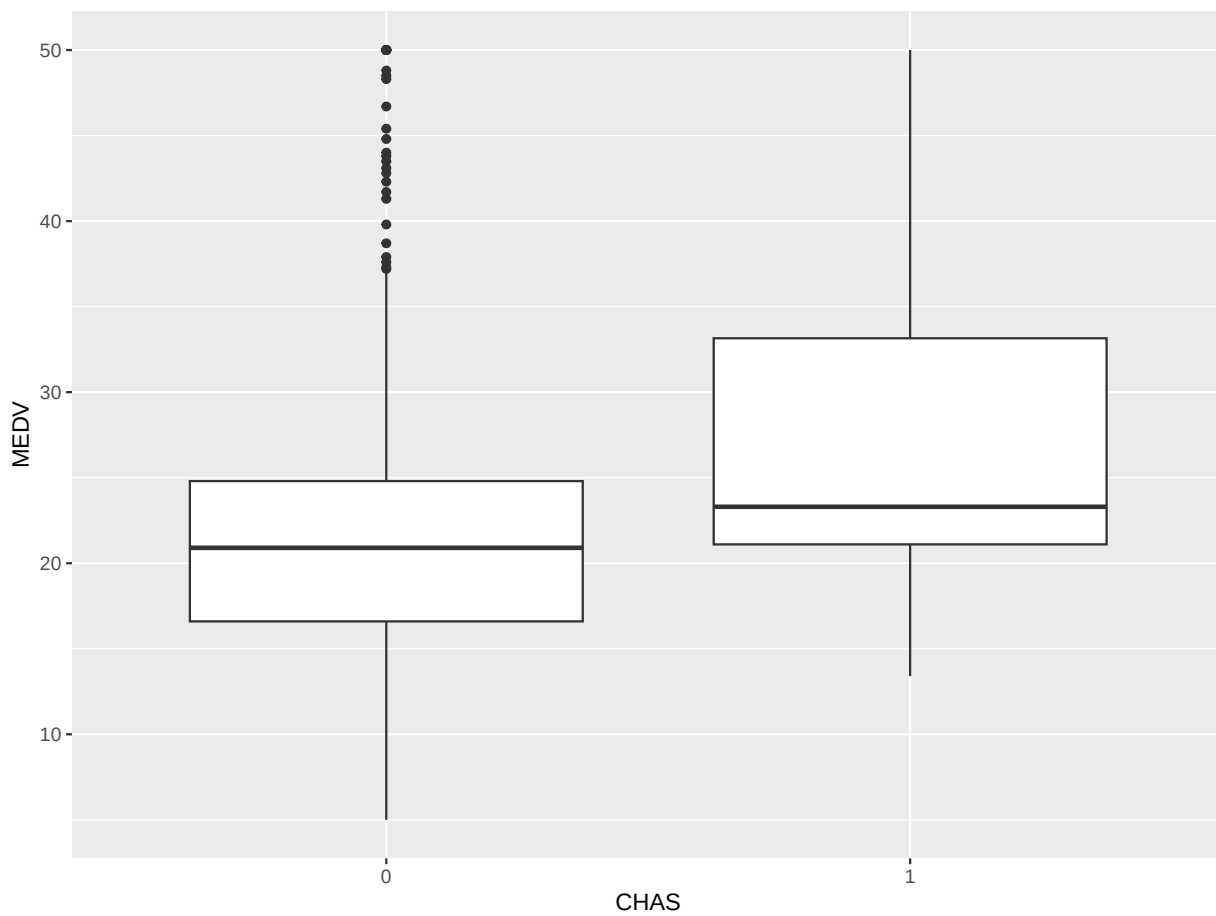
45 **boxplot of MEDV for different values of CHAS**

```
boxplot(housing.df$MEDV ~ housing.df$CHAS, xlab = "CHAS", ylab = "MEDV")
```



46

```
# alternative plot with ggplot  
ggplot(housing.df) + geom_boxplot(aes(x = as.factor(CHAS), y = MEDV)) + xlab("CHAS")
```

**Figure 3.3****side-by-side boxplots**

```
# use par() to split the plots into panels.  
par(mfcol = c(1, 4))  
boxplot(housing.df$NOX ~ housing.df$CAT..MEDV, xlab = "CAT.MEDV", ylab = "NOX")  
boxplot(housing.df$LSTAT ~ housing.df$CAT..MEDV, xlab = "CAT.MEDV", ylab = "LSTAT")  
boxplot(housing.df$PTRATIO ~ housing.df$CAT..MEDV, xlab = "CAT.MEDV", ylab = "PTRATIO")  
boxplot(housing.df$INDUS ~ housing.df$CAT..MEDV, xlab = "CAT.MEDV", ylab = "INDUS")
```

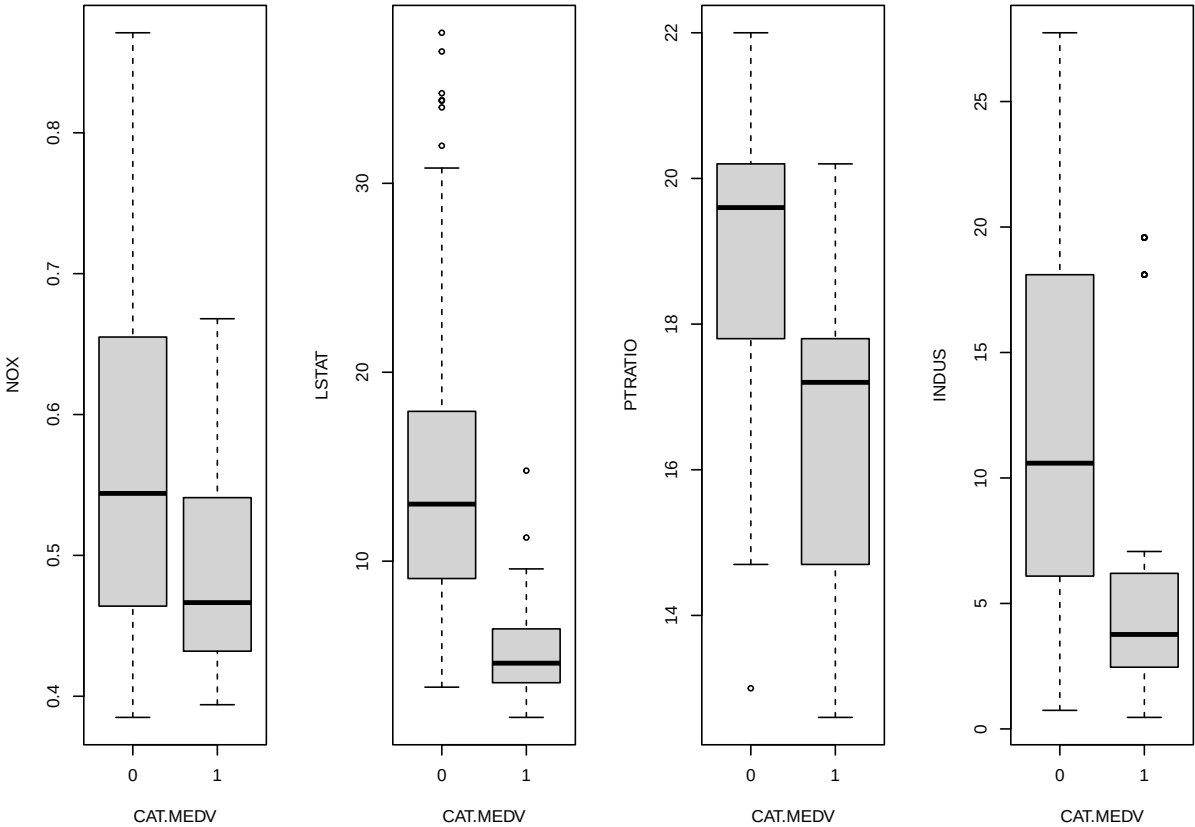
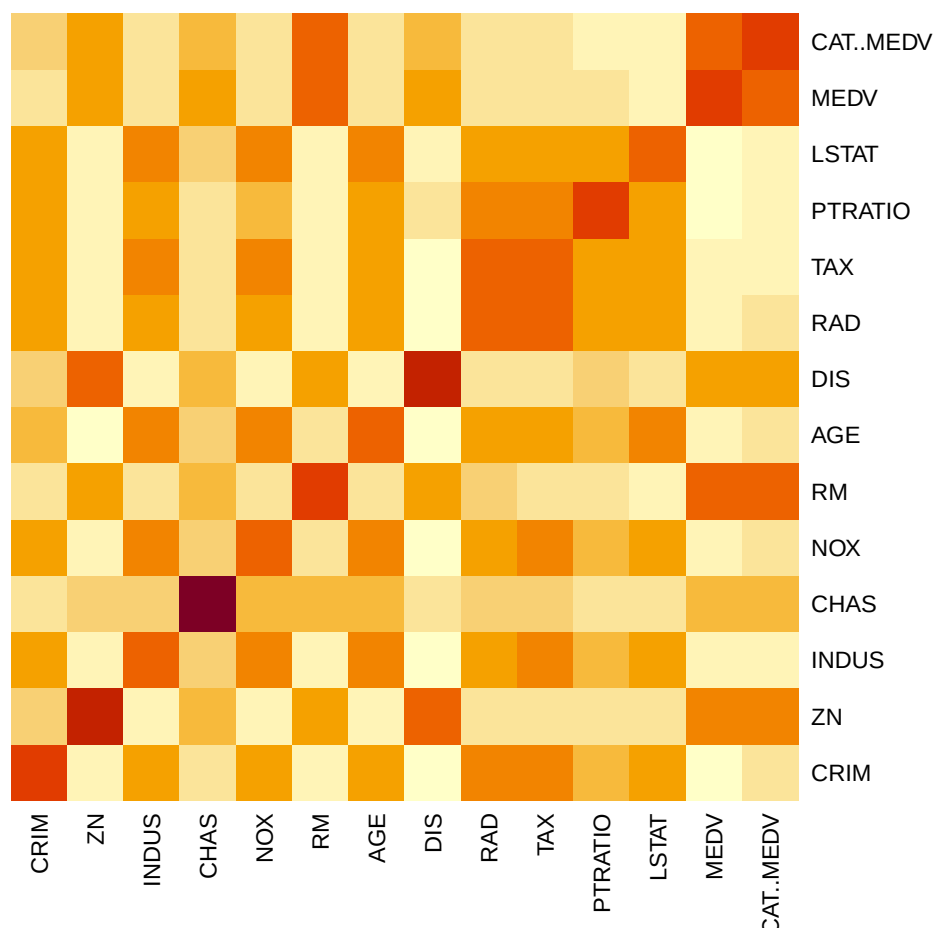


Figure 3.4

simple heatmap of correlations (without values)

```
heatmap(cor(housing.df), Rowv = NA, Colv = NA)
```



53

54 **heatmap with values**

```
library(gplots)
```

55

```
Attaching package: 'gplots'
```

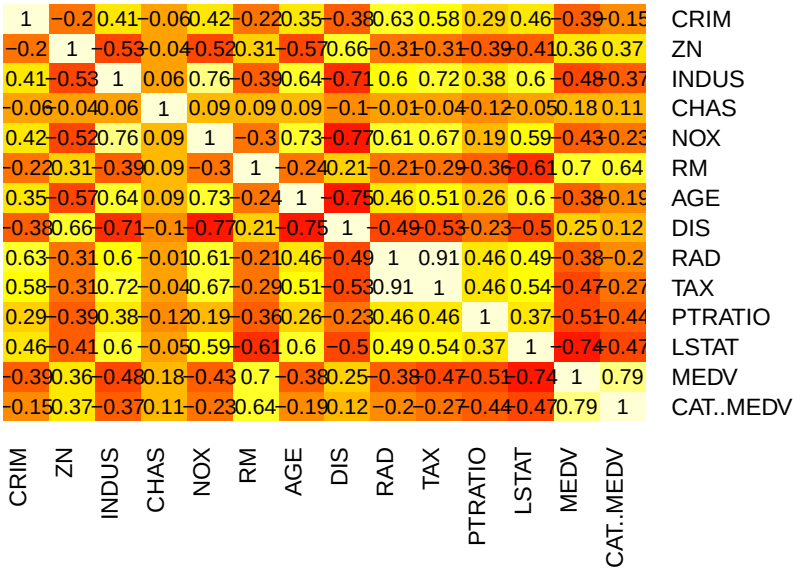
```
The following object is masked from 'package:stats':
```

58

```
lowess
```

59

```
heatmap.2(cor(housing.df), Rowv = FALSE, Colv = FALSE, dendrogram = "none",
  cellnote = round(cor(housing.df),2),
  notecol = "black", key = FALSE, trace = 'none', margins = c(10,10))
```



60

61 **alternative plot with ggplot**

```
library(ggplot2)
library(reshape) # to generate input for the plot
```

62

63 Attaching package: 'reshape'

64 The following object is masked from 'package:lubridate':

65

66 stamp

67 The following object is masked from 'package:dplyr':

68

69 rename

The following objects are masked from 'package:tidyr':

expand, smiths

```
cor.mat <- round(cor(housing.df),2) # rounded correlation matrix
melted.cor.mat <- melt(cor.mat)
ggplot(melted.cor.mat, aes(x = X1, y = X2, fill = value)) +
  geom_tile() +
  geom_text(aes(x = X1, y = X2, label = value))
```

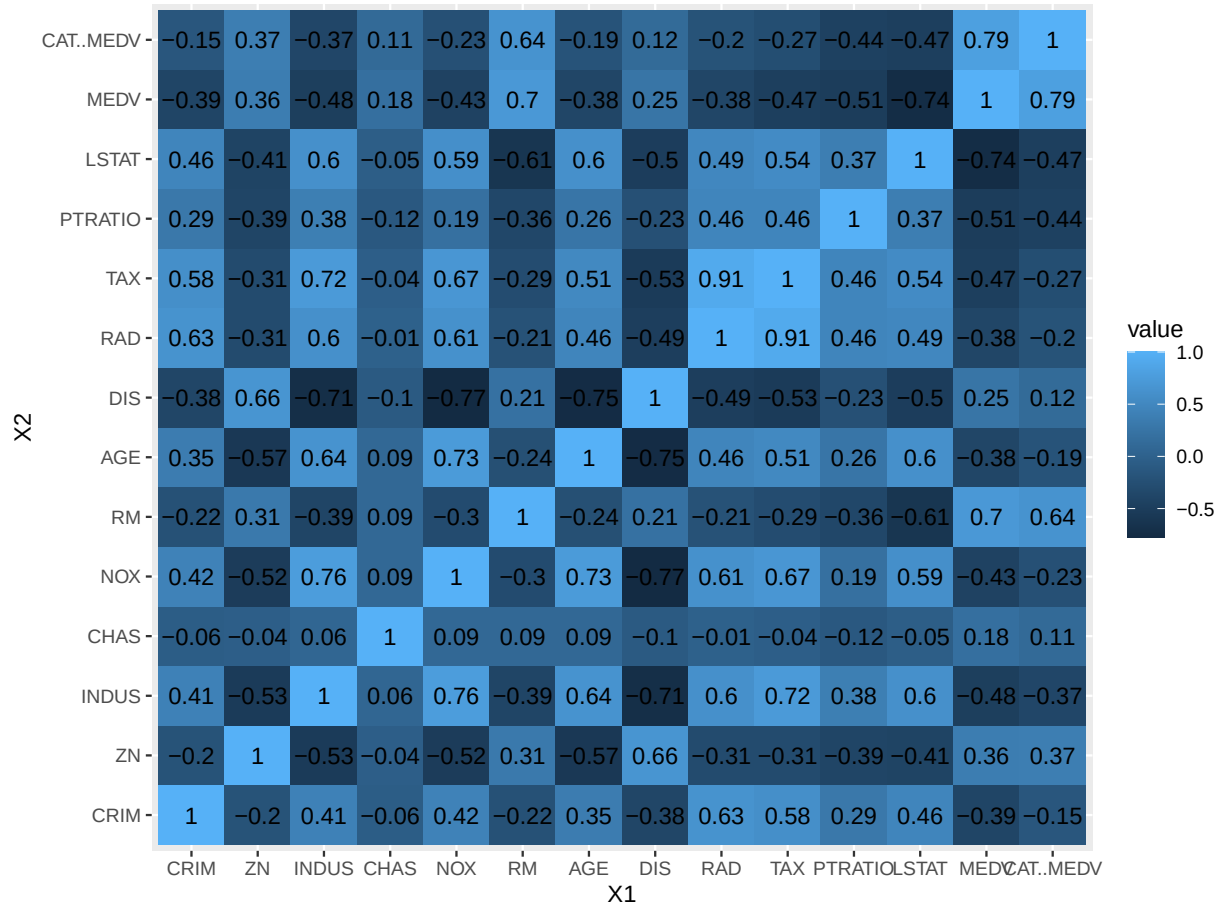


Figure 3.5

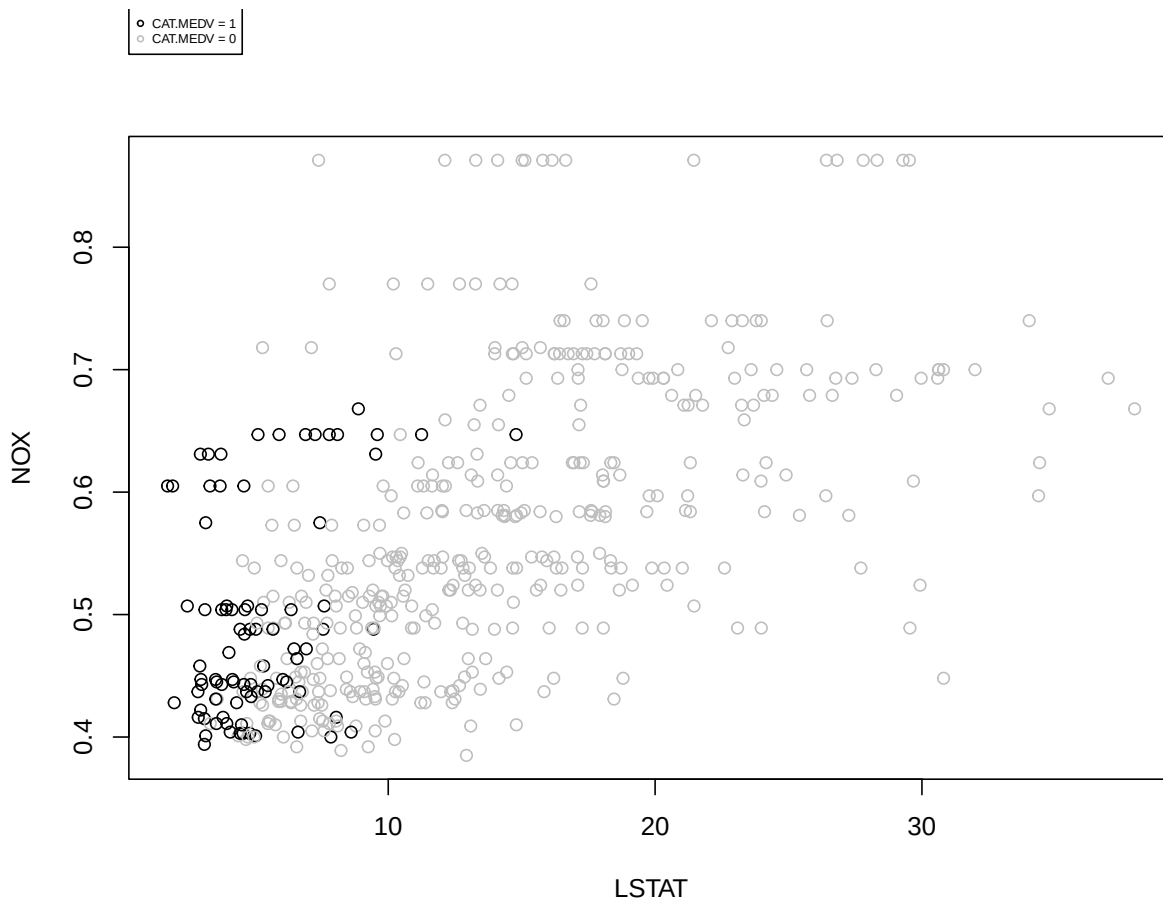
```
# replace dataFrame with your data.
# is.na() returns a Boolean (TRUE/FALSE) output indicating the location of missing
# values.
```

```
# multiplying the Boolean value by 1 converts the output into binary (0/1).  
#heatmap(1 * is.na(dataFrame), Rowv = NA, Colv = NA)
```

75 **Figure 3.6**

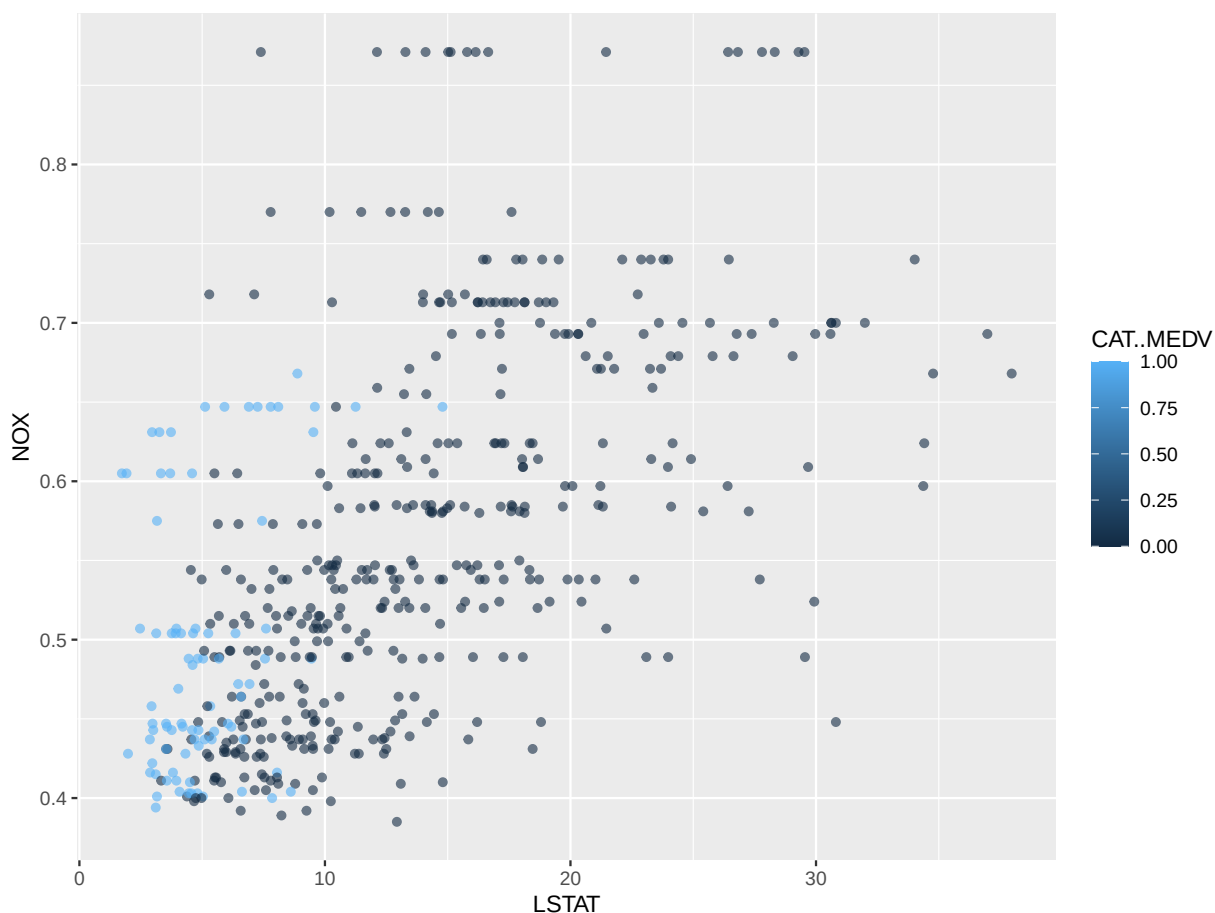
76 **Housing data scatter plot with color CAT..MEDV**

```
## color plot  
par(mfcol = c(1,1), xpd=TRUE) # allow legend to be displayed outside of plot area  
plot(housing.df$NOX ~ housing.df$LSTAT, ylab = "NOX", xlab = "LSTAT",  
     col = ifelse(housing.df$CAT..MEDV == 1, "black", "gray"))  
# add legend outside of plotting area  
# In legend() use argument inset = to control the location of the legend relative  
# to the plot.  
legend("topleft", inset=c(0, -0.2),  
      legend = c("CAT.MEDV = 1", "CAT.MEDV = 0"), col = c("black", "gray"),  
      pch = 1, cex = 0.5)
```

77

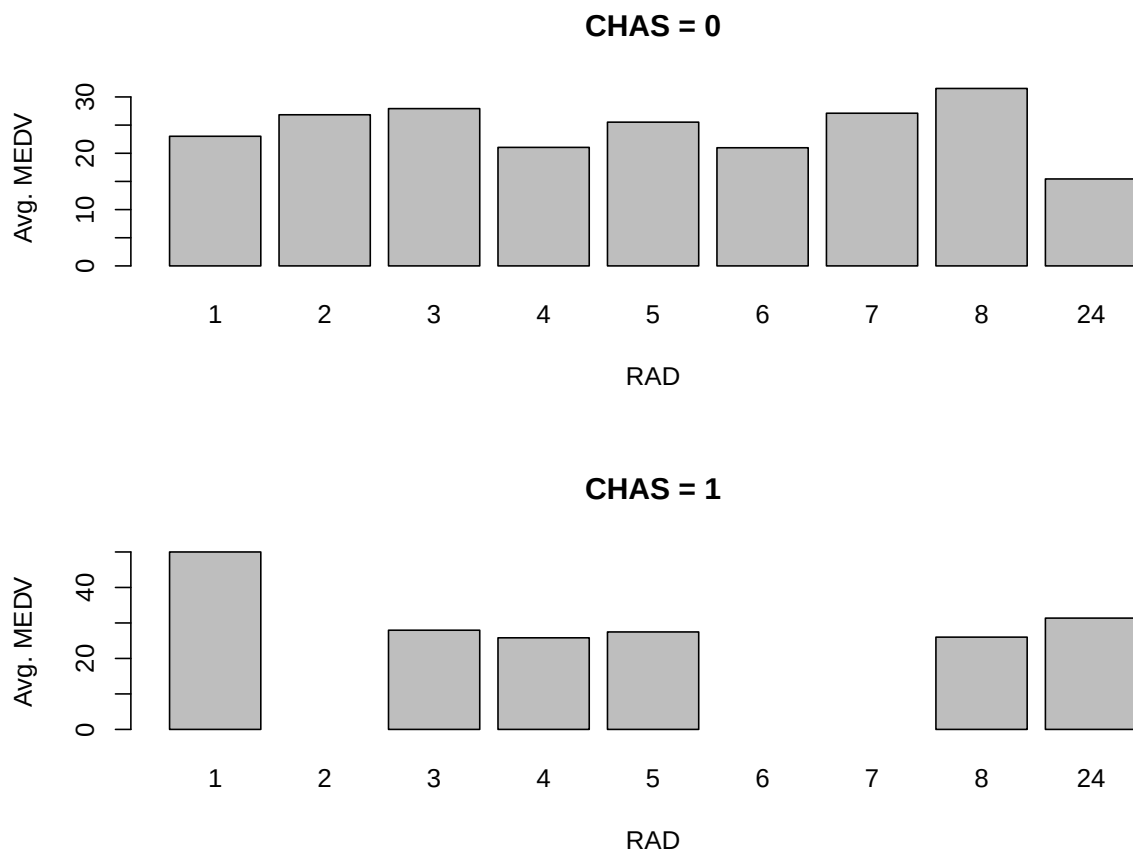
```
# alternative plot with ggplot
library(ggplot2)
ggplot(housing.df, aes(y = NOX, x = LSTAT, colour= CAT..MEDV)) +
  geom_point(alpha = 0.6)
```



78

79 panel plots

```
# compute mean MEDV per RAD and CHAS
# In aggregate() use argument drop = FALSE to include all combinations
# (existing and missing) of RAD X CHAS.
data.for.plot <- aggregate(housing.df$MEDV, by = list(housing.df$RAD, housing.df$CHAS),
                           FUN = mean, drop = FALSE)
names(data.for.plot) <- c("RAD", "CHAS", "meanMEDV")
# plot the data
par(mfcol = c(2,1))
barplot(height = data.for.plot$meanMEDV[data.for.plot$CHAS == 0],
        names.arg = data.for.plot$RAD[data.for.plot$CHAS == 0],
        xlab = "RAD", ylab = "Avg. MEDV", main = "CHAS = 0")
barplot(height = data.for.plot$meanMEDV[data.for.plot$CHAS == 1],
        names.arg = data.for.plot$RAD[data.for.plot$CHAS == 1],
        xlab = "RAD", ylab = "Avg. MEDV", main = "CHAS = 1")
```



80

```
# alternative plot with ggplot
ggplot(data.for.plot) +
  geom_bar(aes(x = as.factor(RAD), y = `meanMEDV`), stat = "identity") +
  xlab("RAD") + facet_grid(CHAS ~ .)
```

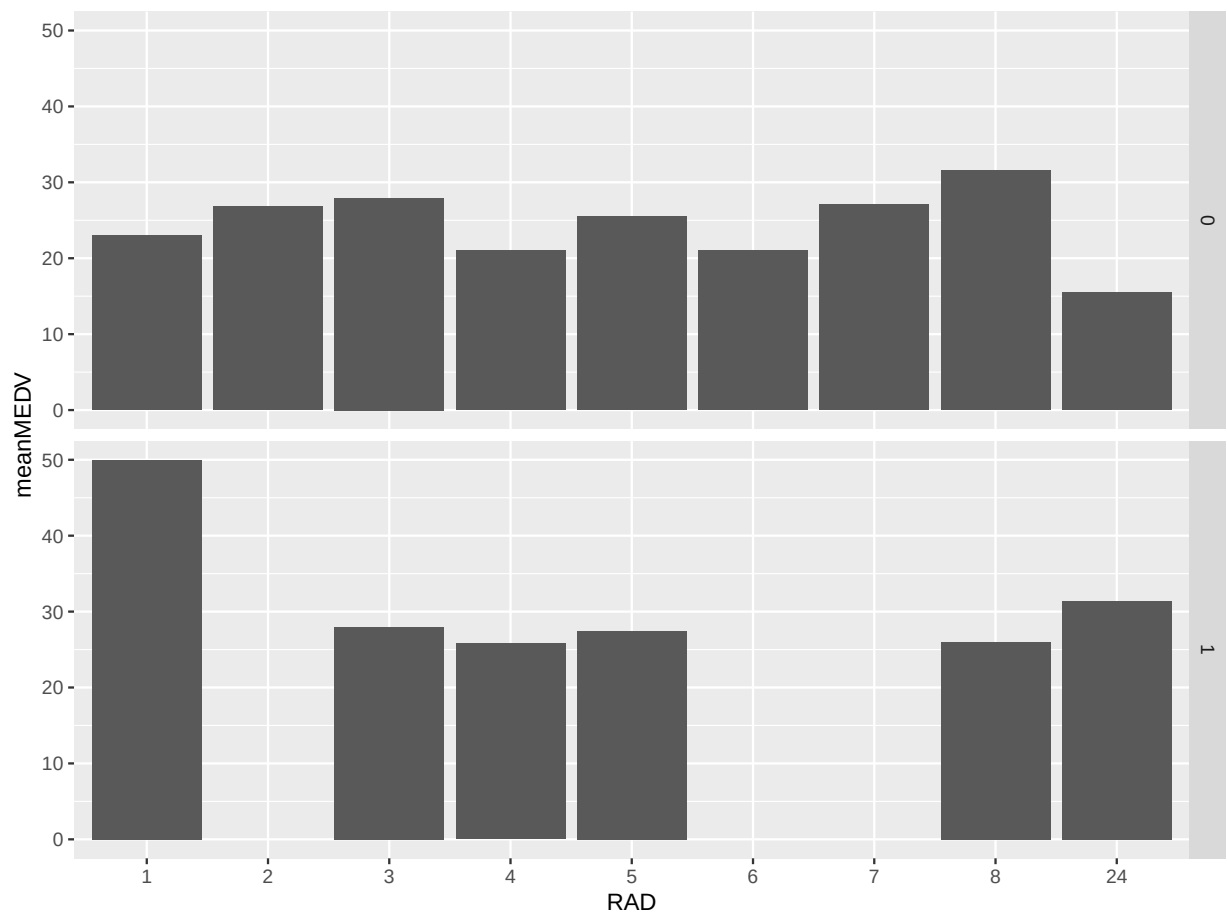
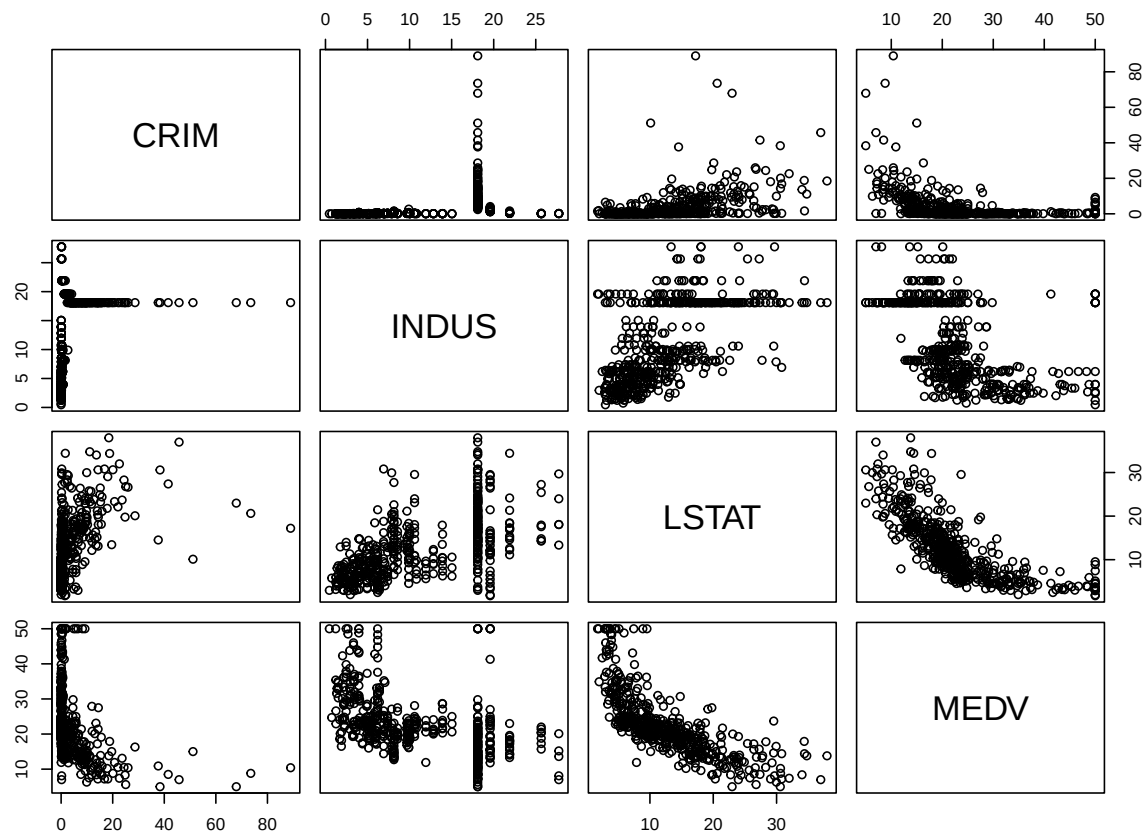


Figure 3.7

simple plot

```
# use plot() to generate a matrix of 4X4 panels with variable name on the diagonal,  
# and scatter plots in the remaining panels.  
plot(housing.df[, c(1, 3, 12, 13)])
```



84

85 **alternative, nicer plot (displayed)**

```
library(GGally)
```

86 Registered S3 method overwritten by 'GGally':

87 method from

88 +.gg ggplot2

```
ggpairs(housing.df[, c(1, 3, 12, 13)])
```

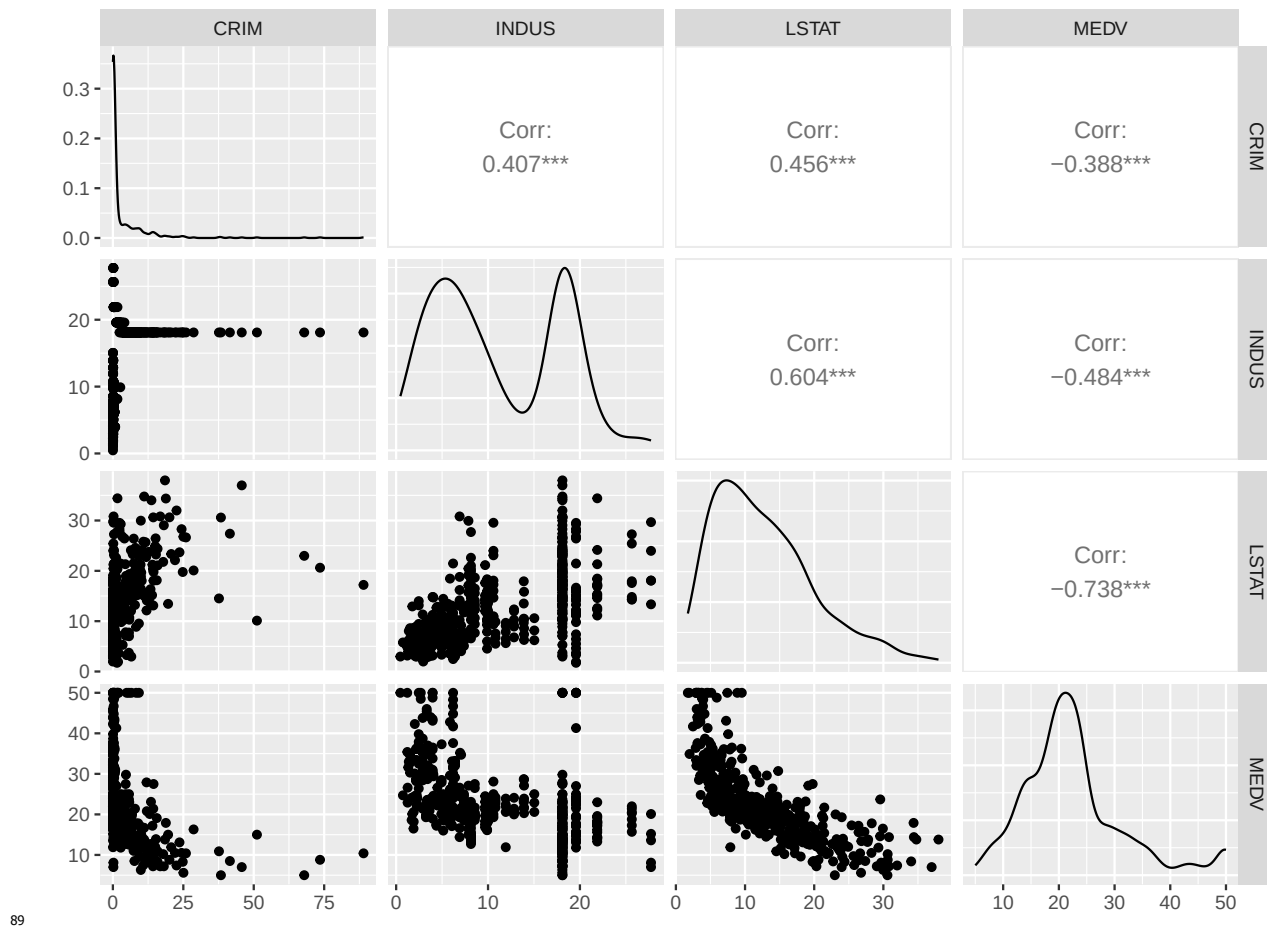
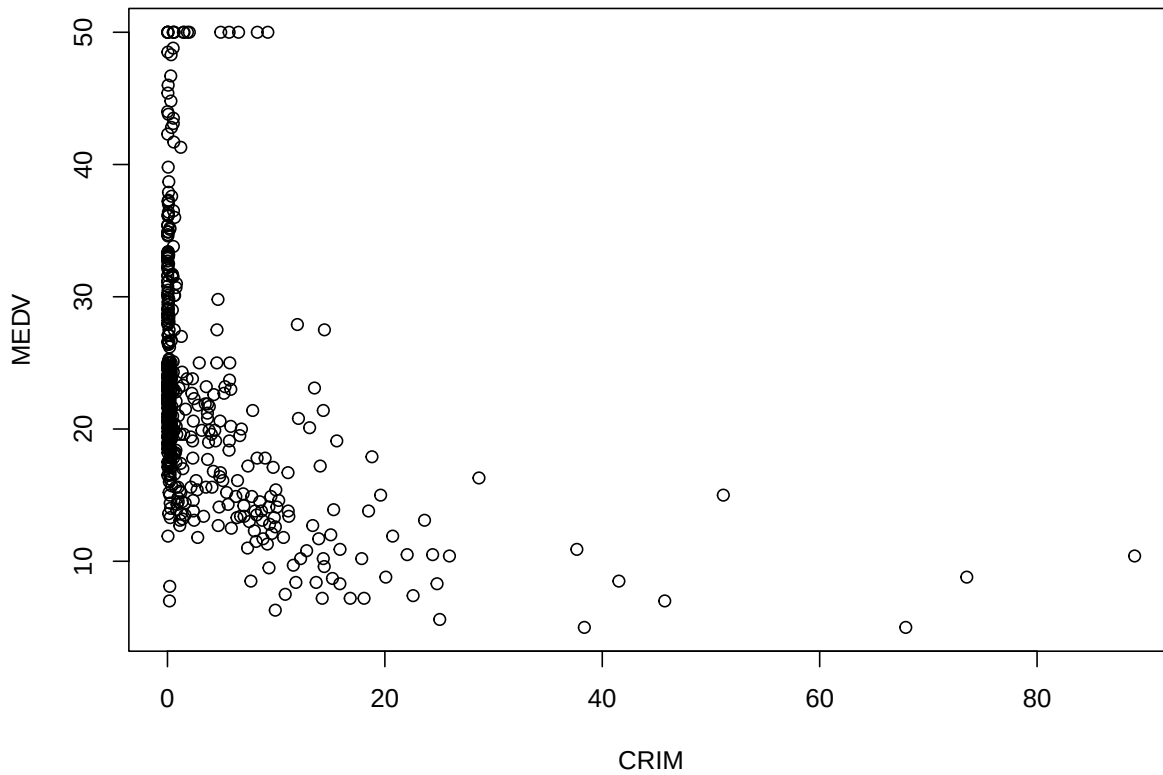


Figure 3.8

Simple plot MEDV CRIM

```
options(scipen=999) # avoid scientific notation

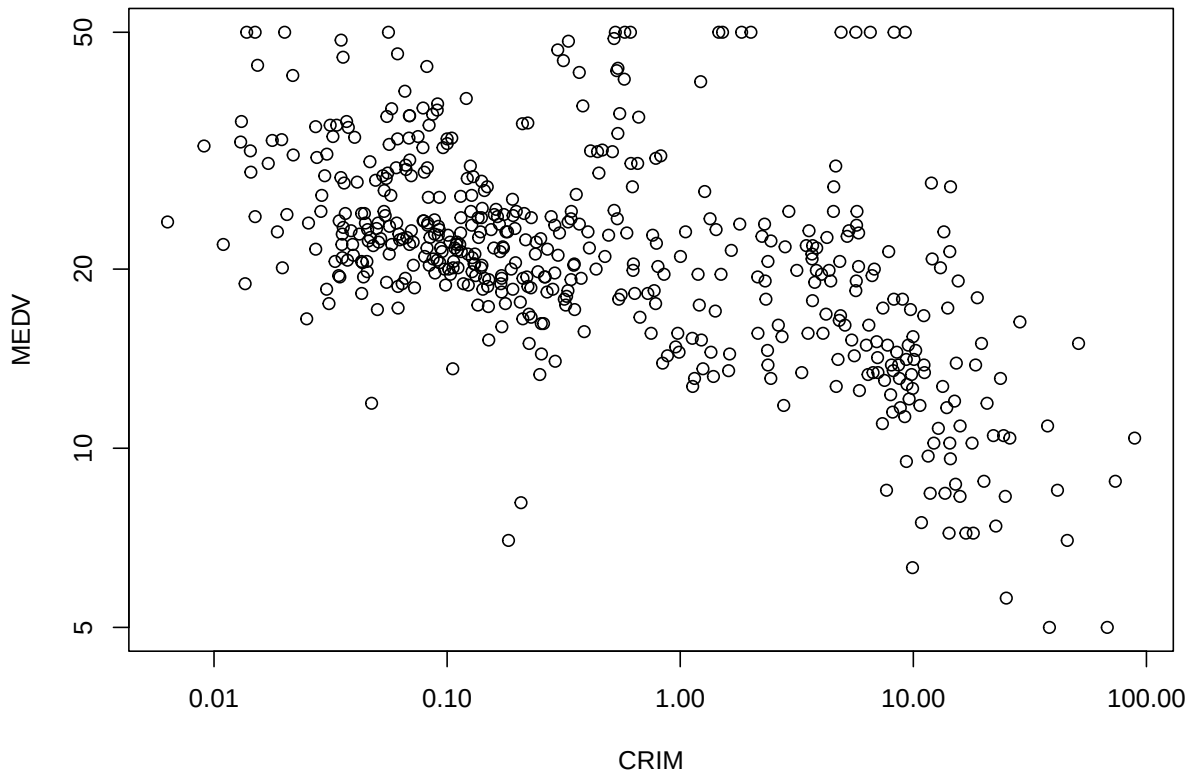
## scatter plot: regular and log scale
plot(housing.df$MEDV ~ housing.df$CRIM, xlab = "CRIM", ylab = "MEDV")
```



92

93 **to use logarithmic scale set argument `log =` to either `'x'`, `'y'`, or `'xy'`.**

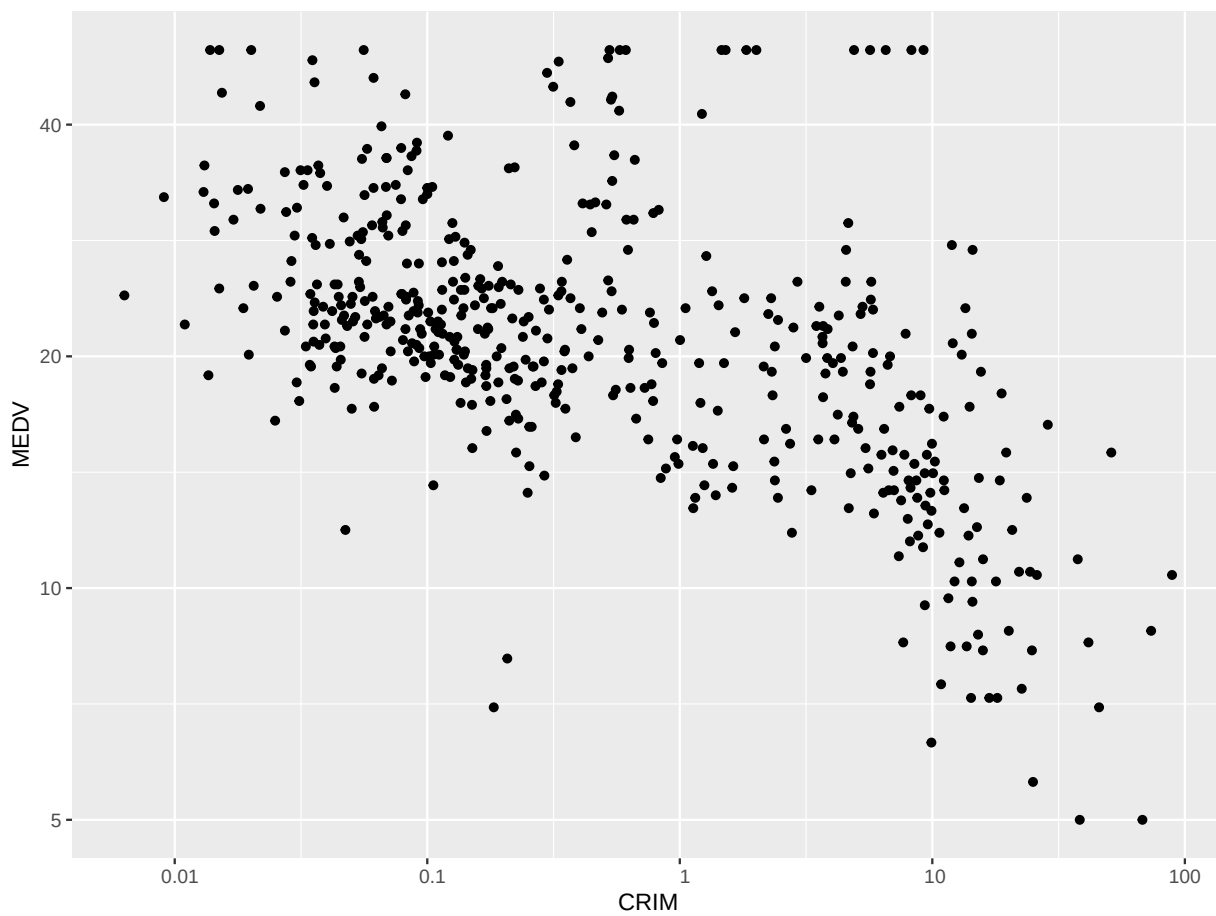
```
plot(housing.df$MEDV ~ housing.df$CRIM,  
     xlab = "CRIM", ylab = "MEDV", log = 'xy')
```



94

95 alternative log-scale plot with ggplot

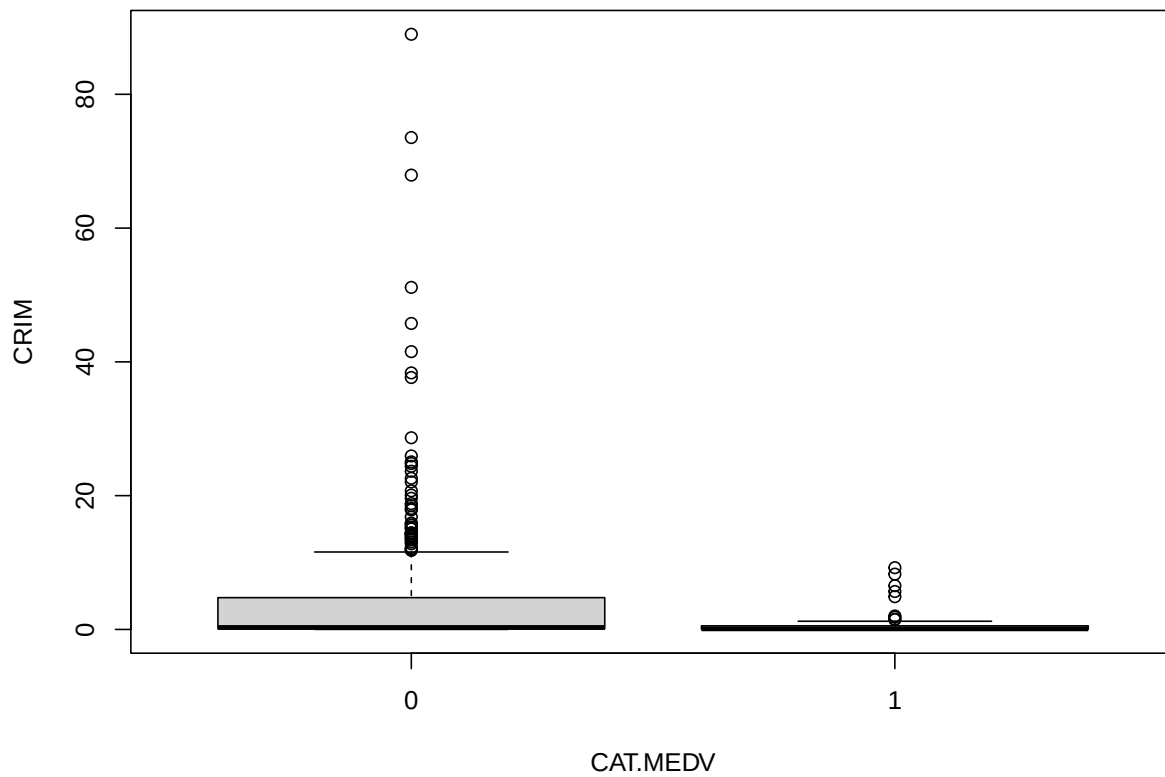
```
library(ggplot2)
ggplot(housing.df) + geom_point(aes(x = CRIM, y = MEDV)) +
  scale_x_log10(breaks = 10^(-2:2),
               labels = format(10^(-2:2), scientific = FALSE, drop0trailing = TRUE)) +
  scale_y_log10(breaks = c(5, 10, 20, 40))
```

96

97 **boxplot: regular and log scale**

```
boxplot(housing.df$CRIM ~ housing.df$CAT..MEDV,  
        xlab = "CAT.MEDV", ylab = "CRIM")
```



98

```
boxplot(housing.df$CRIM ~ housing.df$CAT..MEDV,  
        xlab = "CAT.MEDV", ylab = "CRIM", log = 'y')
```

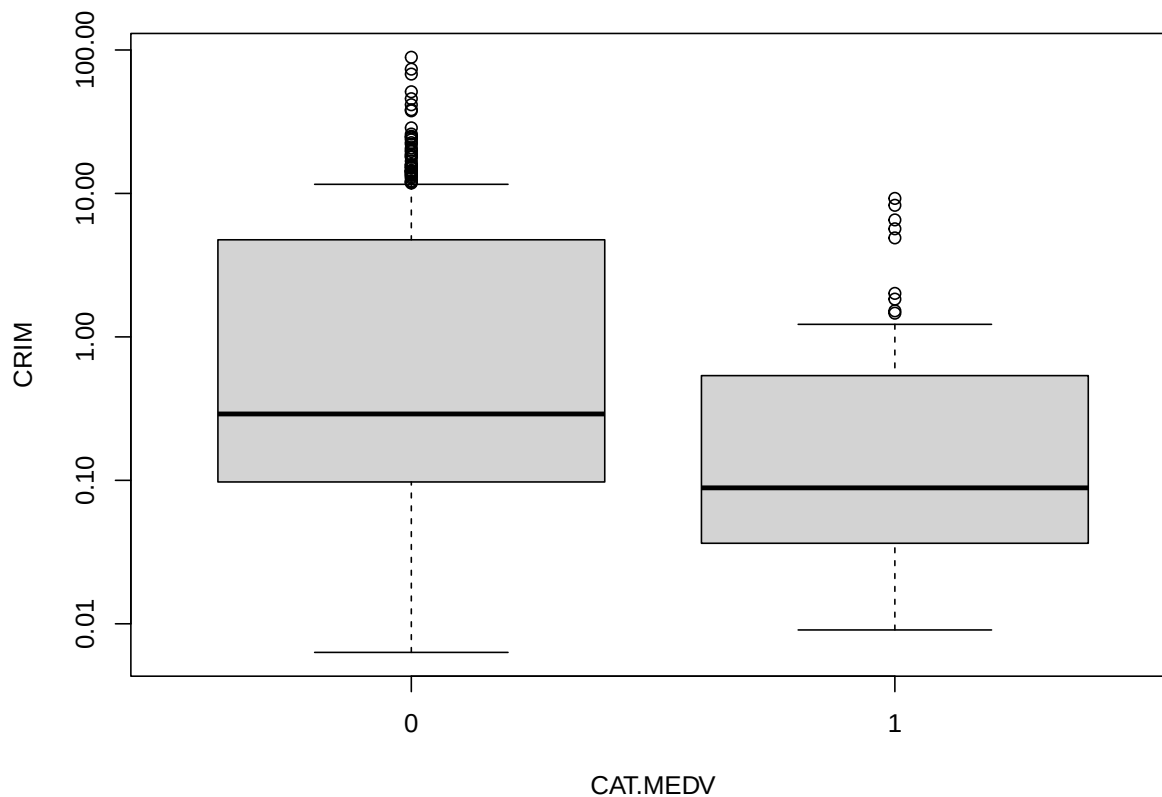
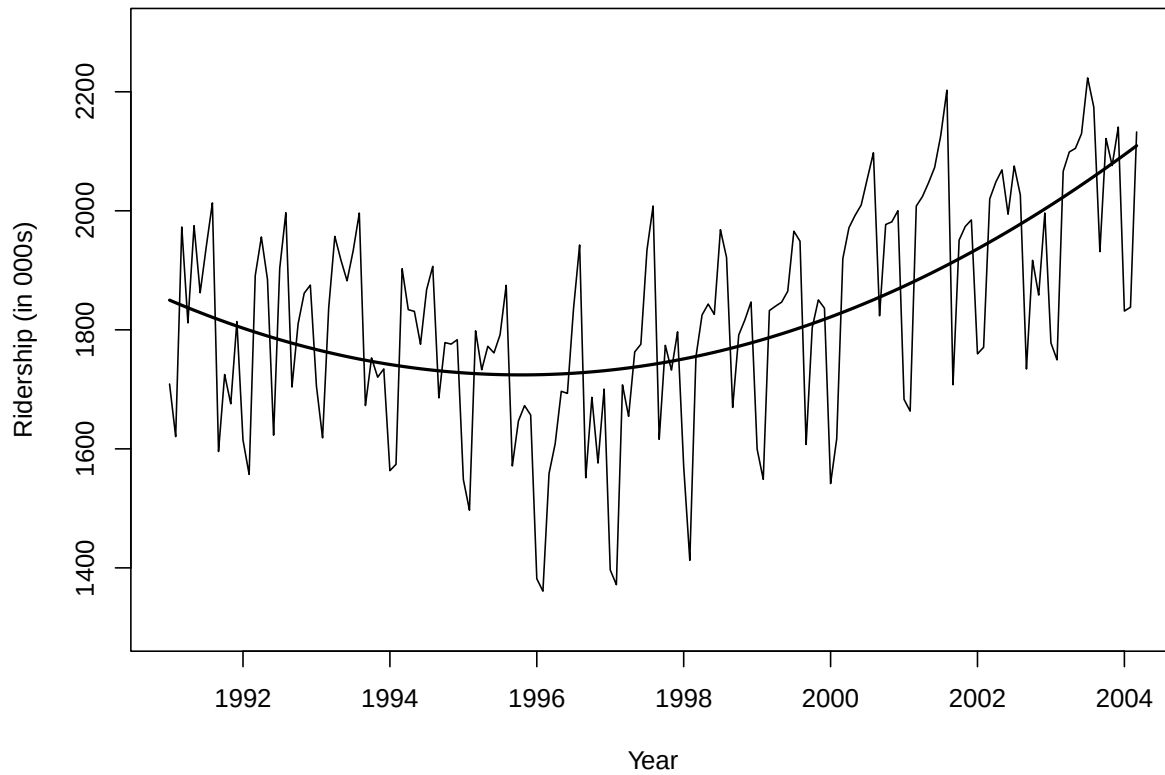


Figure 3.9

```
library(forecast)
Amtrak.df <- read.csv("data/Amtrak data.csv")
ridership.ts <- ts(Amtrak.df$Ridership, start = c(1991, 1), end = c(2004, 3), freq = 12)
```

fit curve

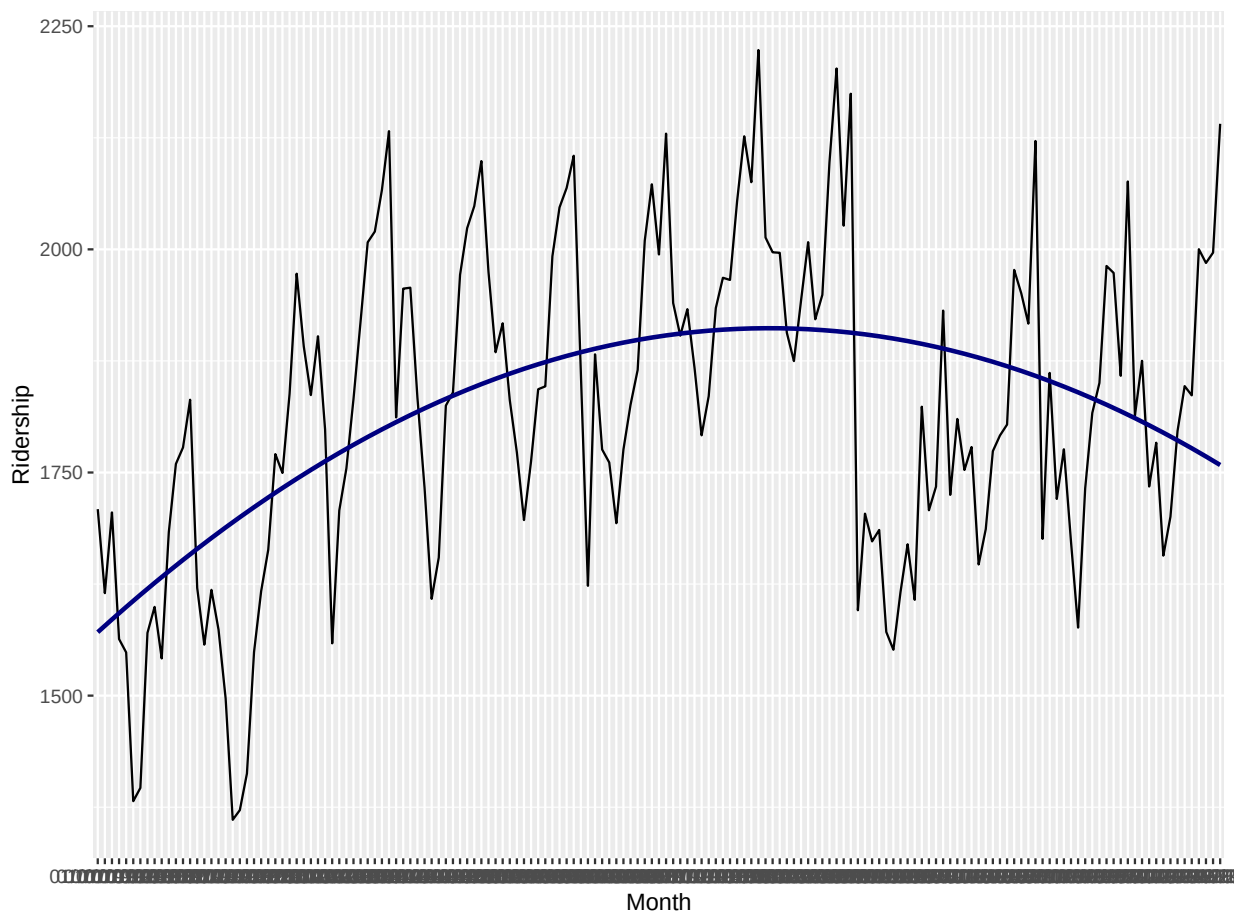
```
ridership.lm <- tslm(ridership.ts ~ trend + I(trend^2))
plot(ridership.ts, xlab = "Year", ylab = "Ridership (in 000s)", ylim = c(1300, 2300))
lines(ridership.lm$fitted, lwd = 2)
```



102

103 **alternative plot with ggplot**

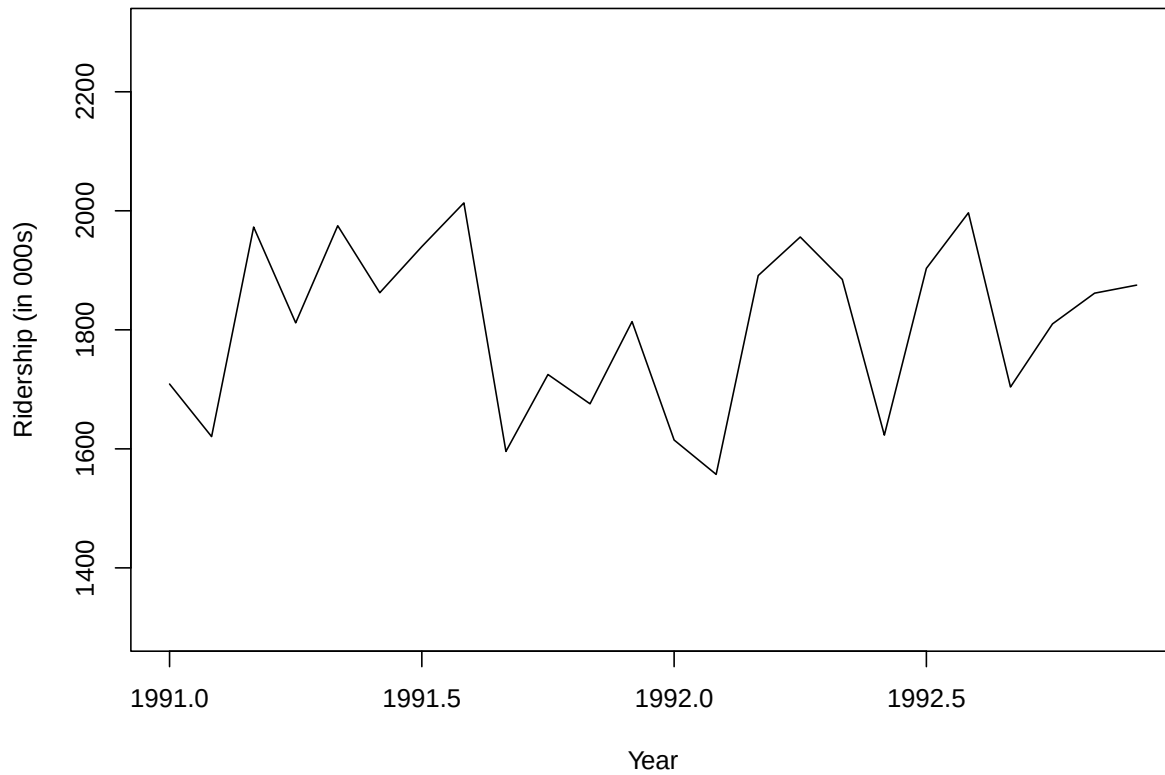
```
library(ggplot2)
ggplot(Amtrak.df, aes(y = Ridership, x = Month, group = 12)) +
  geom_line() + geom_smooth(formula = y ~ poly(x, 2), method= "lm",
                           colour = "navy", se = FALSE, na.rm = TRUE)
```



104

105 **Year**

```
ridership.2yrs <- window(ridership.ts, start = c(1991,1), end = c(1992,12))  
plot(ridership.2yrs, xlab = "Year", ylab = "Ridership (in 000s)", ylim = c(1300, 2300))
```

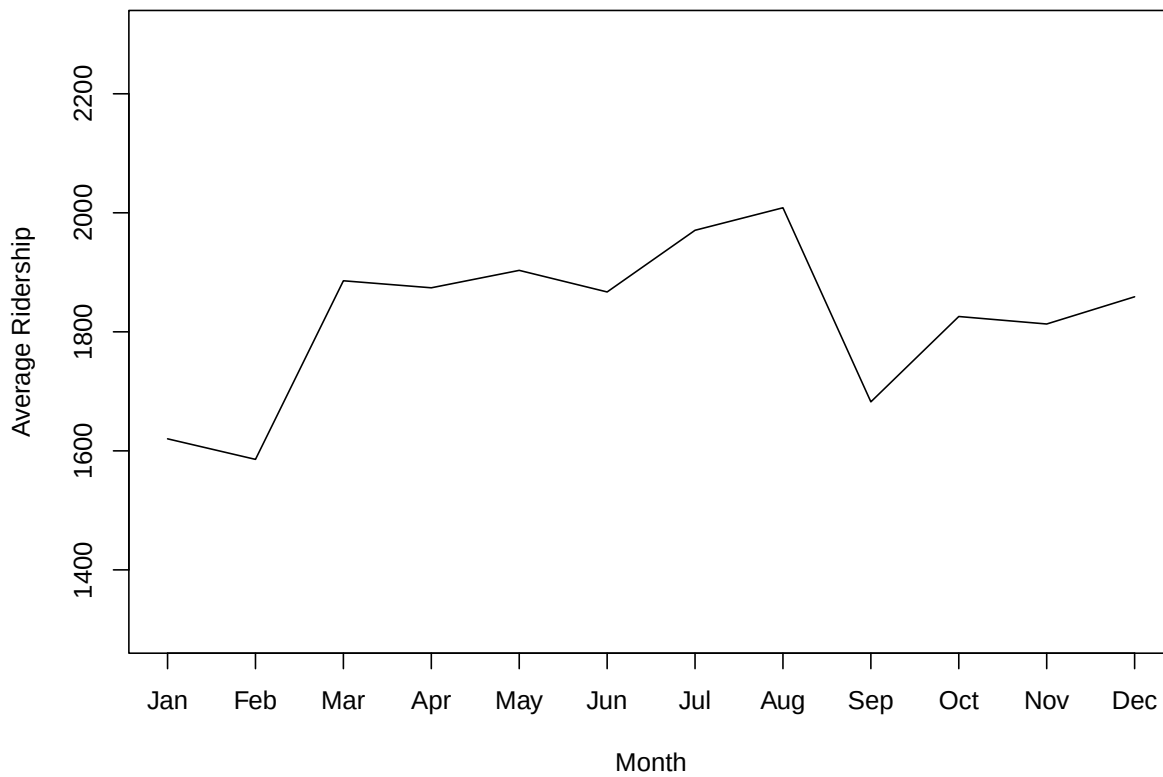


106

Month

107

```
monthly.ridership.ts <- tapply(ridership.ts, cycle(ridership.ts), mean)
plot(monthly.ridership.ts, xlab = "Month", ylab = "Average Ridership",
     ylim = c(1300, 2300), type = "l", xaxt = 'n')
## set x labels
axis(1, at = c(1:12), labels = c("Jan", "Feb", "Mar", "Apr", "May", "Jun",
                                "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"))
```



108

109 **Year(agggregated)**

```
annual.ridership.ts <- aggregate(ridership.ts, FUN = mean)
plot(annual.ridership.ts, xlab = "Year", ylab = "Average Ridership",
     ylim = c(1300, 2300))
```

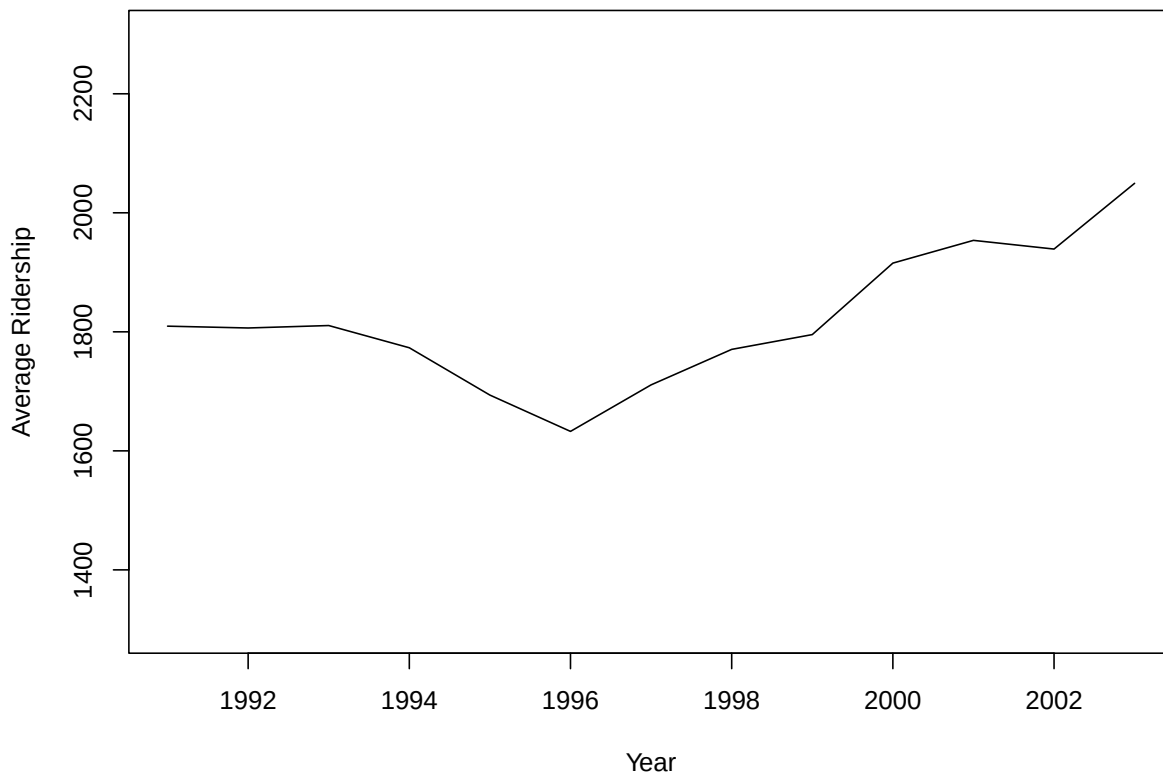
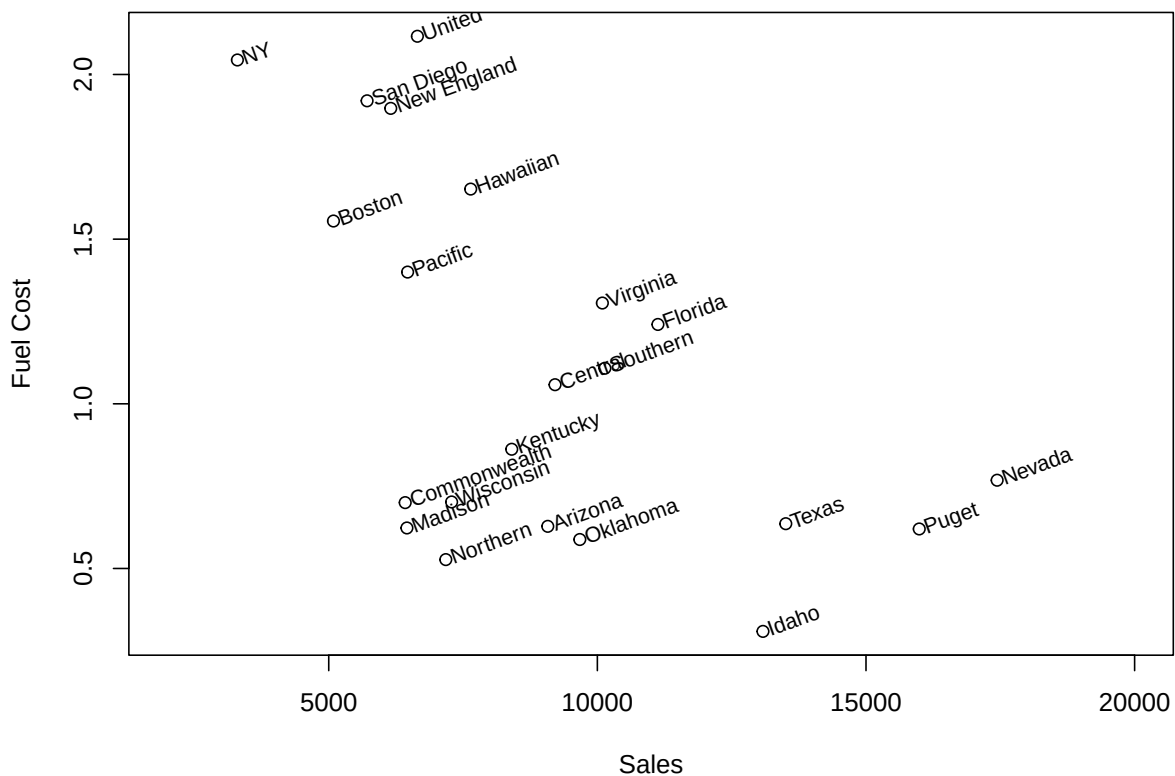


Figure 3.10

Utilities sales

```
utilities.df <- read.csv("data/Utilities.csv")

plot(utilities.df$Fuel_Cost ~ utilities.df$Sales,
     xlab = "Sales", ylab = "Fuel Cost", xlim = c(2000, 20000))
text(x = utilities.df$Sales, y = utilities.df$Fuel_Cost,
     labels = utilities.df$Company, pos = 4, cex = 0.8, srt = 20, offset = 0.2)
```

113

114 **alternative with ggplot**

```
library(ggplot2)
ggplot(utilities.df, aes(y = Fuel_Cost, x = Sales)) + geom_point() +
  geom_text(aes(label = paste(" ", Company)), size = 4, hjust = 0.0, angle = 15) +
  ylim(0.25, 2.25) + xlim(3000, 18000)
```

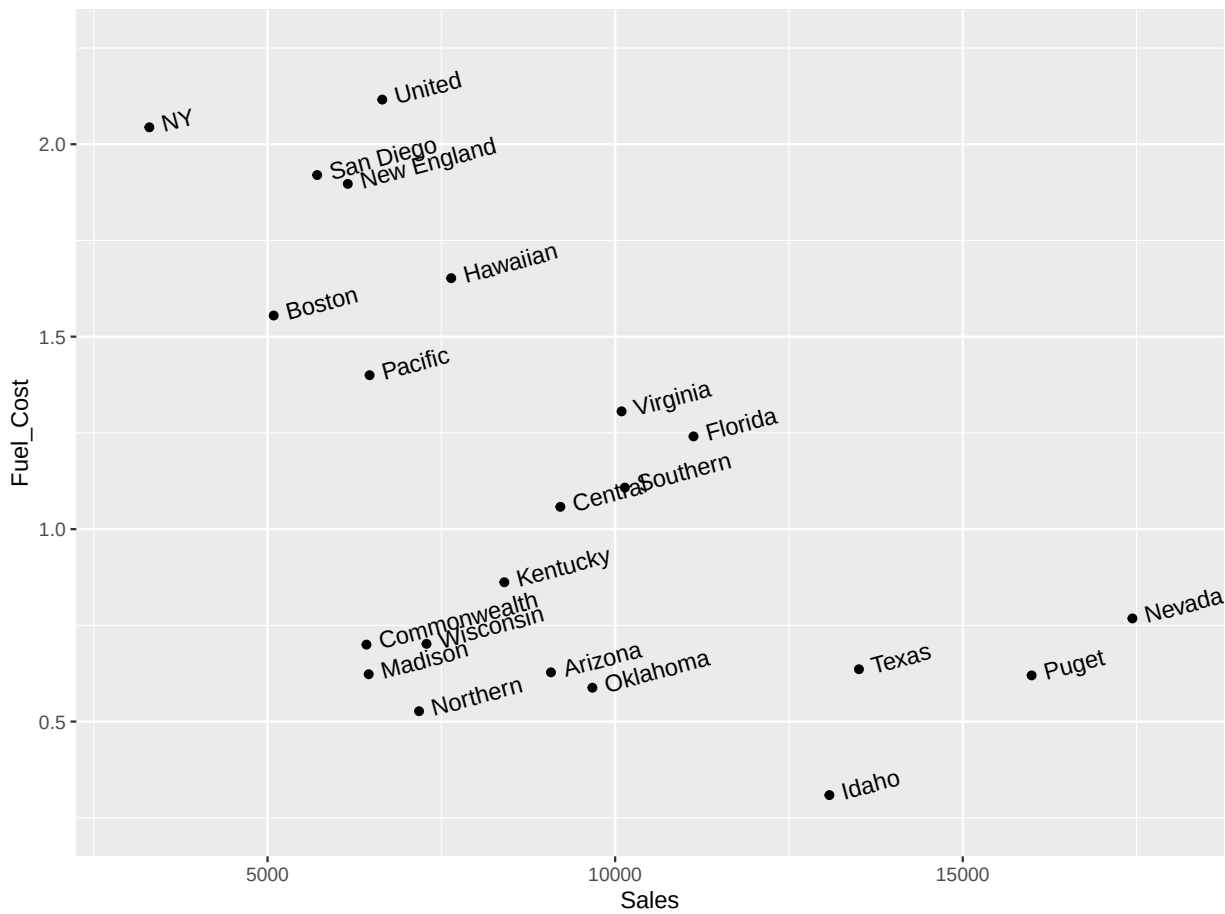


Figure 3.11

use function `alpha()` in library `scales` to add transparent colors

```
# use function alpha() in library scales to add transparent colors
universal.df <- read.csv("data/UniversalBank.csv")

library(scales)
```

Attaching package: 'scales'

The following object is masked from 'package:purrr':

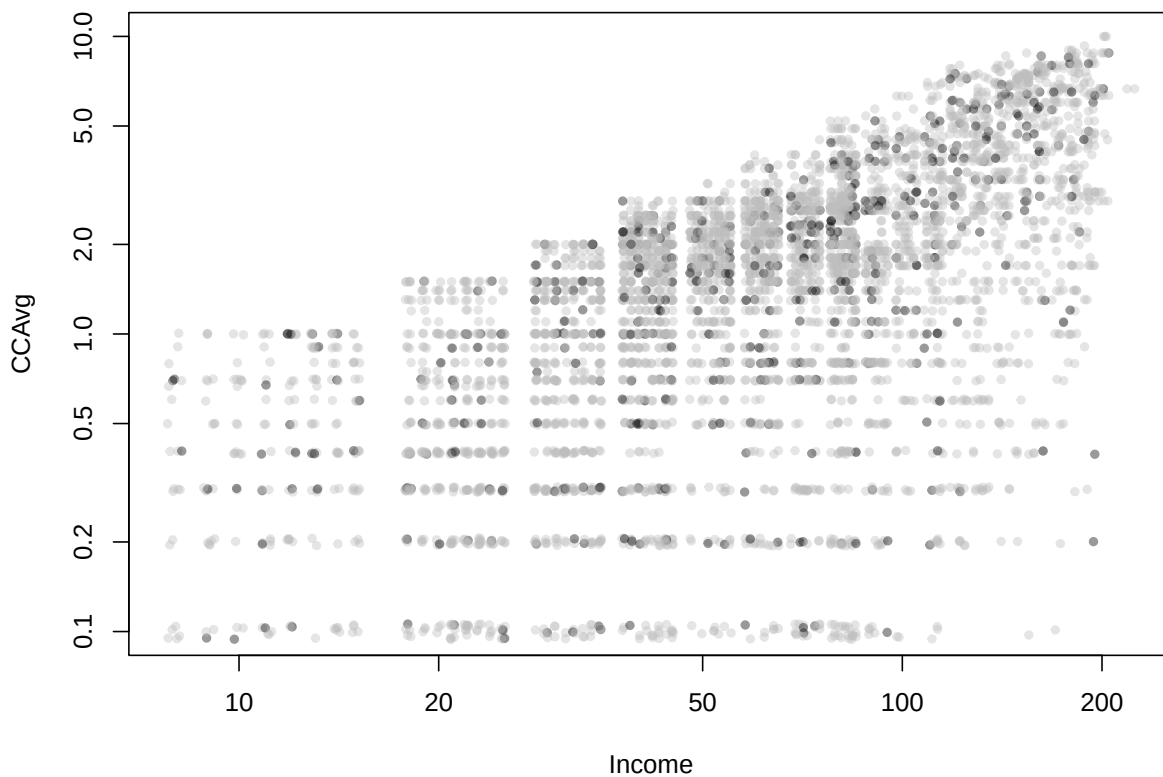
`discard`

123 The following object is masked from 'package:readr':

124

125 col_factor

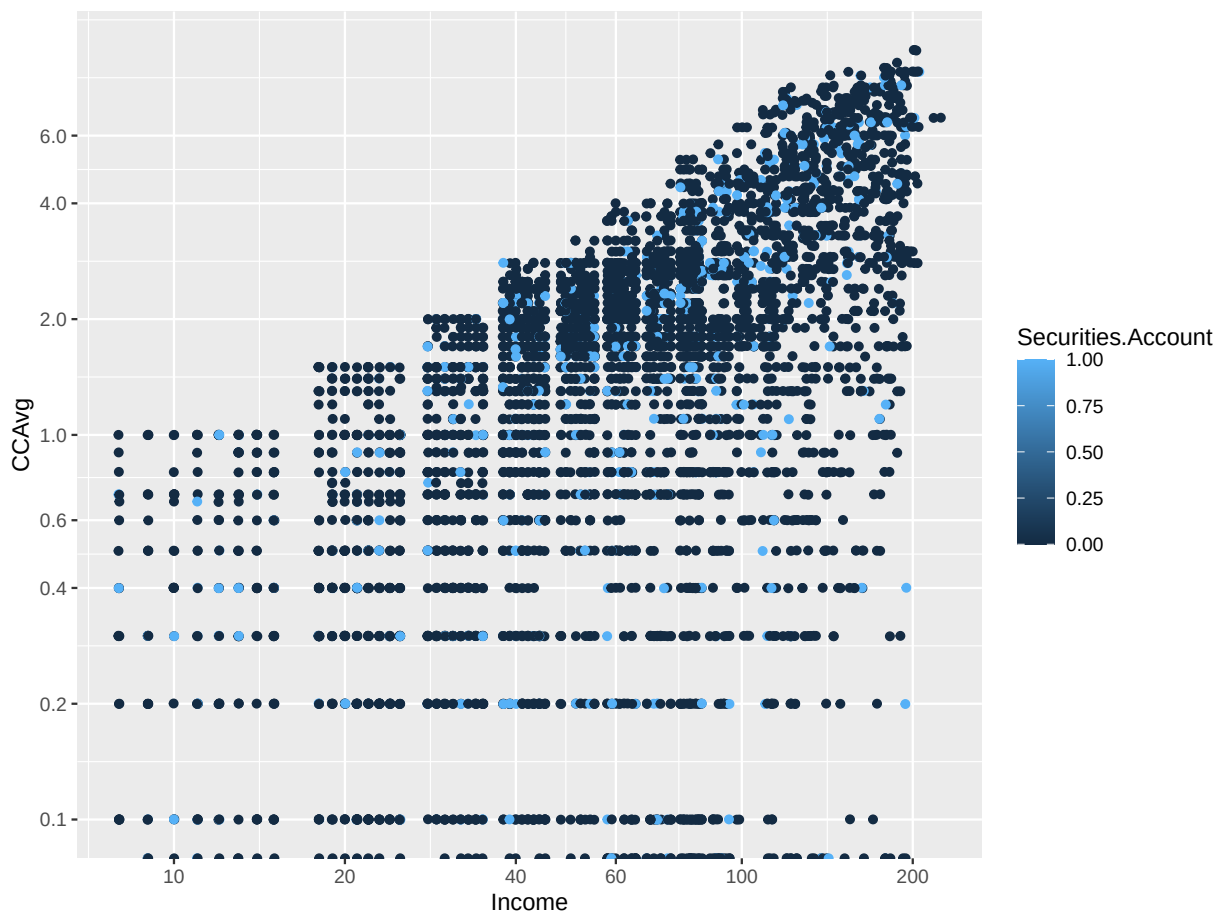
```
plot(jitter(universal.df$CCAvg, 1) ~ jitter(universal.df$Income, 1),
     col = alpha(ifelse(universal.df$Securities.Account == 0, "gray", "black"), 0.4),
     pch = 20, log = 'xy', ylim = c(0.1, 10),
     xlab = "Income", ylab = "CCAvg")
```



126

127 **alternative with ggplot**

```
library(ggplot2)
ggplot(universal.df) +
  geom_jitter(aes(x = Income, y = CCAvg, colour = Securities.Account)) +
  scale_x_log10(breaks = c(10, 20, 40, 60, 100, 200)) +
  scale_y_log10(breaks = c(0.1, 0.2, 0.4, 0.6, 1.0, 2.0, 4.0, 6.0))
```

**Figure 3.12****Paracoord plot**

```
library(MASS)
```

```
Attaching package: 'MASS'
```

```
The following object is masked from 'package:dplyr':
```

```
select
```

```
par(mfcol = c(2,1))
parcoord(housing.df[housing.df$CAT..MEDV == 0, -14], main = "CAT.MEDV = 0")
parcoord(housing.df[housing.df$CAT..MEDV == 1, -14], main = "CAT.MEDV = 1")
```

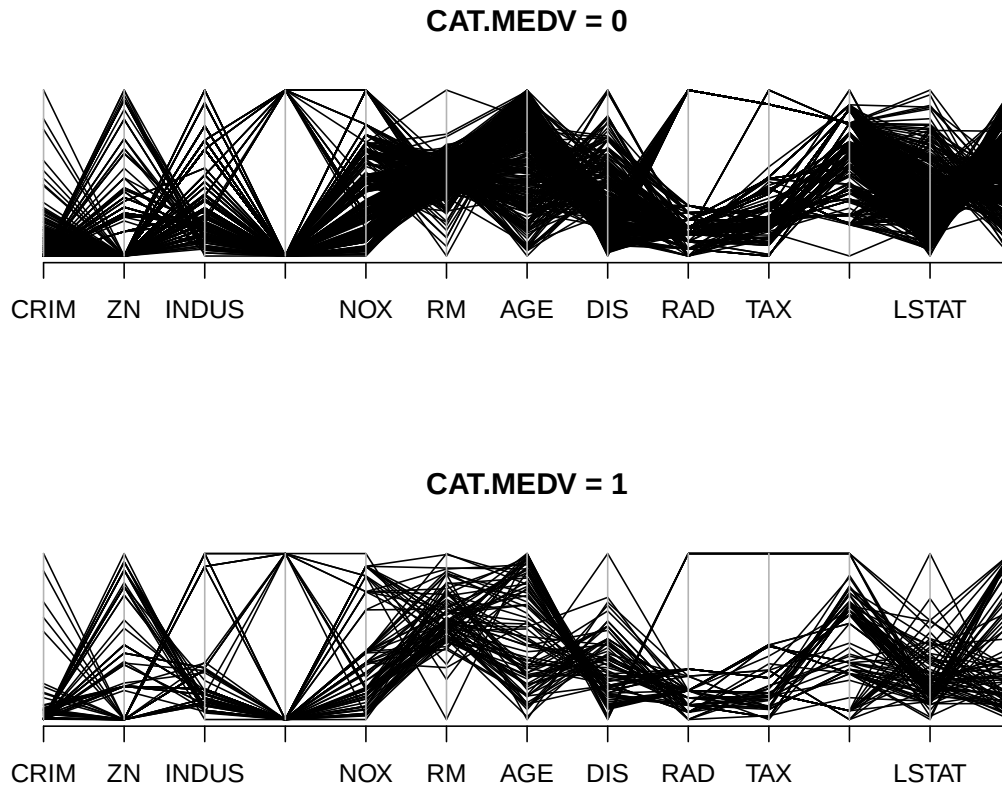


Figure 3.14

Network

```
library(igraph)
```

Attaching package: 'igraph'

141 The following objects are masked from 'package:lubridate':

142
143 %--%, union

144 The following objects are masked from 'package:dplyr':

145
146 as_data_frame, groups, union

147 The following objects are masked from 'package:purrr':

148
149 compose, simplify

150 The following object is masked from 'package:tidyr':

151
152 crossing

153 The following object is masked from 'package:tibble':

154
155 as_data_frame

156 The following objects are masked from 'package:stats':

157
158 decompose, spectrum

159 The following object is masked from 'package:base':

160
161 union

```
ebay.df <- read.csv("data/eBayNetwork.csv")

# transform node ids to factors
ebay.df[,1] <- as.factor(ebay.df[,1])
ebay.df[,2] <- as.factor(ebay.df[,2])

graph.edges <- as.matrix(ebay.df[,1:2])
g <- graph.edgelist(graph.edges, directed = FALSE)
isBuyer <- V(g)$name %in% graph.edges[,2]

plot(g, vertex.label = NA, vertex.color = ifelse(isBuyer, "gray", "black"),
     vertex.size = ifelse(isBuyer, 7, 10))
```

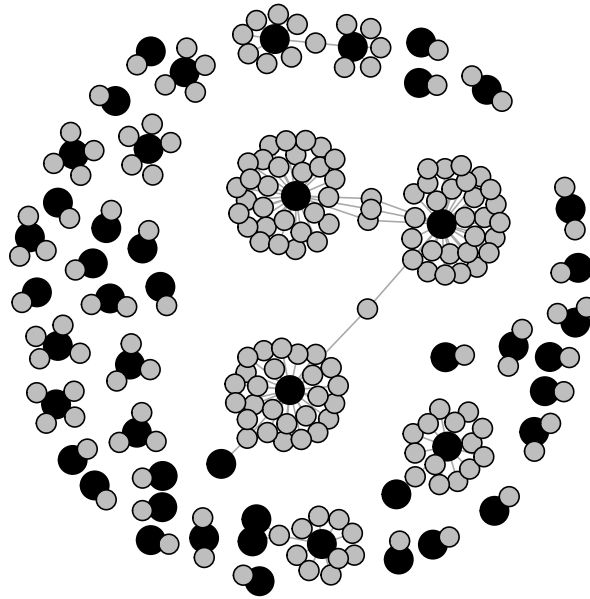


Figure 3.15

Treemap

```
library(treemap)
tree.df <- read.csv("data/EbayTreemap.csv")

# add column for negative feedback
tree.df$negative.feedback <- 1* (tree.df$Seller.Feedback < 0)

# draw treemap
treemap(tree.df, index = c("Category","Sub.Category", "Brand"),
        vSize = "High.Bid", vColor = "negative.feedback", fun.aggregate = "mean",
        align.labels = list(c("left", "top"), c("right", "bottom"), c("center", "center")),
        palette = rev(gray.colors(3)), type = "manual", title = "")
```

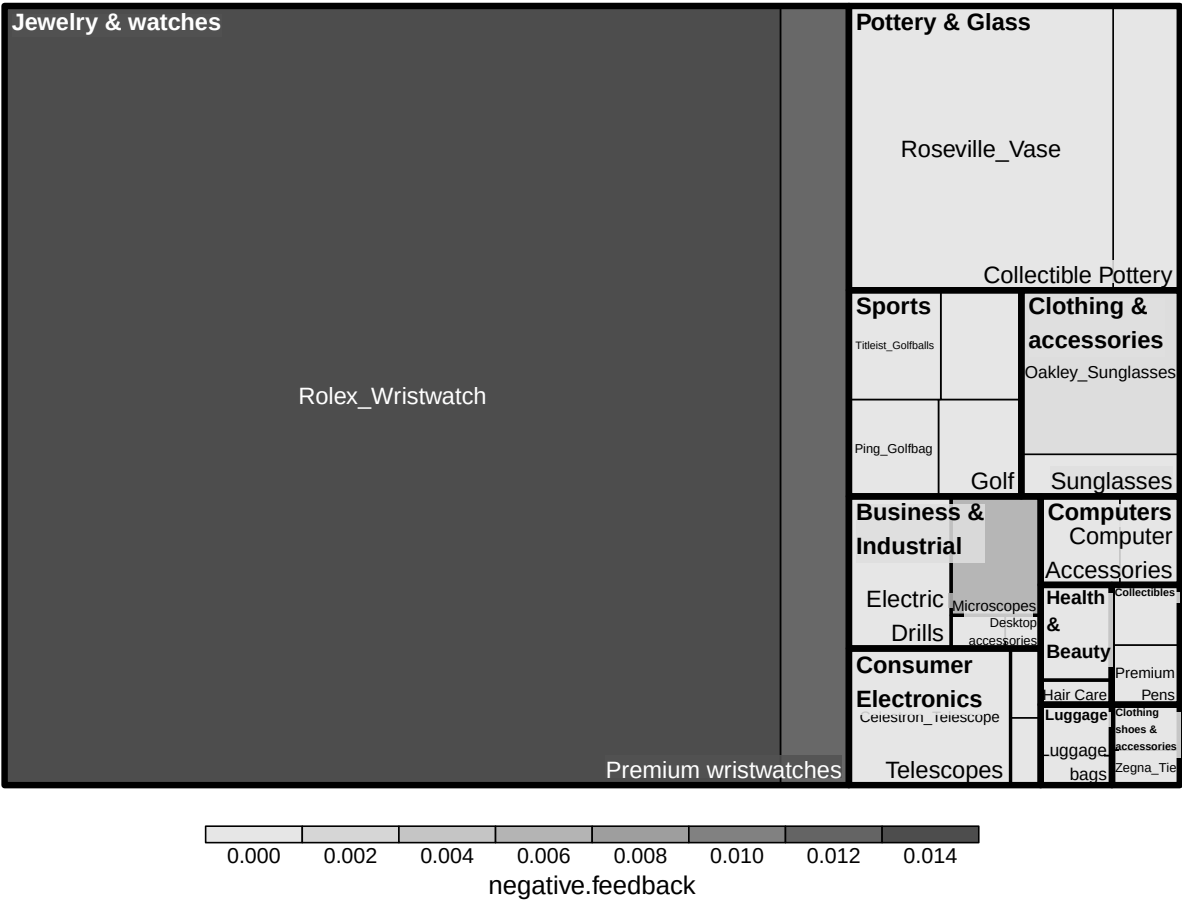


Figure 3.16

Map1

```
#library(ggmap)
#SCstudents <- read.csv("data/SC-US-students-GPS-data-2016.csv")
#Map <- get_map("Denver, CO", zoom = 3)
#ggmap(Map) + geom_point(aes(x = longitude, y = latitude), data = SCstudents,
#                          alpha = 0.4, colour = "red", size = 0.5)
```


Figure 3.17**Map2**

```
#library(mosaic)
#
#gdp.df <- read.csv("data/gdp.csv", skip = 4, stringsAsFactors = FALSE)
#names(gdp.df)[5] <- "GDP2015"
#happiness.df <- read.csv("data/Veerhoven.csv")
#
# gdp map
#mWorldMap(gdp.df, key = "Country.Name", fill = "GDP2015") + coord_map()
#
# eell-being map
#mWorldMap(happiness.df, key = "Nation", fill = "Score") + coord_map() +
#  scale_fill_continuous(name = "Happiness")
```